Engineering Applications of Artificial Intelligence 151 (2025) 110589



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai





Design and implementation of a soft Actor–Critic controller for a robotic arm

Ping-Huan Kuo [a,b,*], Chen-Ting Huang [a], Chen-Wen Chang [c], Po-Hsun Feng [c], Yu-Sian Lin [a]

[a] Department of Mechanical Engineering, National Chung Cheng University, Chiayi, 62102, Taiwan
[b] Advanced Institute of Manufacturing with High-tech Innovations (AIM-HI), National Chung Cheng University, Chiayi, 62102, Taiwan
[c] Department of Electrical Engineering, National Cheng Kung University, Tainan, 701401, Taiwan

ARTICLE INFO

Keywords:
Robotic arm controller
Reinforcement learning
Obstacle avoidance
Six-degree-of-freedom robotic arm
Soft actor–critic algorithm

ABSTRACT

In this study, reinforcement learning (RL) was employed to train and control a robotic arm with six degrees of freedom in a point tracking task. The movements of such a robotic arm are not intuitive because they are determined by the rotations of and lengths between its six joints. Given this consideration, RL can be employed for the robotic arm to learn from its mistakes and autonomously explore the effects of joint rotation of its end point position. This approach is mostly used in complex learning tasks, such as those in gaming, robotics, and automated driving. The proposed learning process was verified in a simulator, in which point tracking and obstacle avoidance tasks were performed using RL algorithms. The performance of the soft actor–critic (SAC), proximal policy optimization, advantage actor–critic, and trust region policy optimization algorithms was compared in the point tracking task, and the results indicated that the SAC algorithm outperformed the other algorithms in this task. Therefore, the SAC algorithm was used with an adequately designed reward function to conduct model training, enabling the robotic arm to achieve obstacle avoidance in an object picking task. The established model was verified to allow the robotic arm to learn from failed experiences, enable fast navigation to the target position, and facilitate rapid deployment in future application.

1. Introduction

Industry 4.0 has led to not only higher precision and productivity requirements for common industrial machines and robotic arms but also the development of smart manufacturing through the integration of industrial machines and robotic arms with cloud monitoring, Internet of Things, and big data technologies. The eventual goal of Industry 4.0 is to achieve unmanned factories by replacing human labor completely with robotic arms. Robotic arms have a wide range of applications, including common pick-and-place tasks, S, welding, and microsurgery.

Robotic arms with six or more degrees of freedom (DoFs) are highly agile, and their translation and rotation can be controlled along the *x*-, *y*-, and *z*-axes. Such robotic arms can closely approximate the movements of human arms and thus have a wide range of applications. However, as the number of DoFs increases, the complexity of movement control calculations also increases. This thus means that controlling robotic arms that have six or more DoFs is inherently complex and not intuitive. Reinforcement learning (RL; Groot et al., 2024; X. Li et al., 2024; H. Wang et al., 2024) can facilitate the training of such robotic arms to control the gripper's position and direction. The RL process includes path planning, which is performed in a stepwise manner to guide movement and reduce the likelihood of collisions following inverse kinematic calculations. A robotic arm is typically programmed by calibrating its movement points, thereby allowing it to perform the same action repeatedly by following the same points. Because these points are fixed, numerous jigs are required to fix the workpiece or peripheral machinery, which limits the arm's flexibility. To avoid this problem, the present study developed a model that implements self-exploratory arm control during the training process. This model can enhance the flexibility of the arm and enable smart movements in various scenarios. Moreover, the proposed model can accelerate processes in unmanned factories by enabling robotic arms to perform rapid path planning and reducing the computation time associated with traditional inverse kinematics. This model also lays the foundation for future adaptive control

* Corresponding author. Department of Mechanical Engineering, National Chung Cheng University, Chiayi, 62102, Taiwan.
E-mail addresses: phkuo@ccu.edu.tw (P.-H. Kuo), iris26220402@gmail.com (C.-T. Huang), casper88tw@gmail.com (C.-W. Chang), iphonejblike@icloud.com (P.-H. Feng), james28202159@gmail.com (Y.-S. Lin).

https://doi.org/10.1016/j.engappai.2025.110589
Received 4 October 2024; Received in revised form 27 February 2025; Accepted 13 March 2025
Available online 31 March 2025
0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

in human–robot collaborations, which can make factories smarter. By facilitating and flexible robotic arm control, the proposed model can benefit diversified small-batch production in modern manufacturing.

The application of robotic arms in various fields has increased considerably (Wigren et al., 2020), and they are typically used to replace human labor in dangerous or highly repetitive tasks to improve safety and efficiency (Pasandi et al., 2022). proposed an integrated and programmable central pattern generator that can generate a reference joint trajectory for a robotic system on the basis of the required cyclic motions, which are basic motions in robotics. This generator is composed of a bounded-output programmable oscillator, which enables the generation of a smooth reference joint trajectory that is similar to the desired trajectory while preserving the robot's position and velocity joint limits (Lan et al., 2023).

Robotic arms can be controlled through various models (Saab and Ghanem, 2018). designed a discrete-time multivariable controller for a certain class of robotic arms without using joint velocities; their controller also had tracking ability and produced smooth torque signals. This controller was a proportional–derivative controller with gravity compensation, and its control law involved plausible and partial compensation of the friction and centripetal forces. To ensure stability, the aforementioned authors also developed a suboptimal stochastic controller that assumes the existence of noise in joint position and velocity measurements. Through this controller, tracking errors can be made arbitrarily small when the sampling rate is sufficiently high.

(Shakouri and Assadian, 2022) introduced a class of time-varying controllers for Euler–Lagrange systems to ensure that convergence occurs at an arbitrary finite time independent of the initial conditions and free of chattering. Said scholar developed a controller scheme based on a mapping technique by using two design steps: 1) specifying a conventional (obstacle avoidance) asymptotically stable controller for the nominal system and 2) achieving a prescribed-time (obstacle avoidance) controller for the perturbed system through simple substitution. The aforementioned authors demonstrated that this model is stable when applied to unperturbed systems in uniform prescribed time and is suitable for perturbed systems in prescribed time because it rejects matched disturbances with unknown upper bounds when no disturbances are observed. RL has the potential to solve complex tasks and is commonly used in games (Gafni and Cohen, 2021). explored a class of restless multiarmed bandit problems with unknown arm dynamics. In a game designed by said scholars, players select an arm to be their active arm and receive a random reward from a specific selection of reward states, which vary according to unknown Markovian dynamics. The objective is to create an arm selection strategy that minimizes regret, which is defined as the reward loss relative to the reward obtained by the player that selects the most rewarding arm throughout the game. Gafni and Cohen developed an adaptive sequencing rules algorithm that adaptively updates and controls the sequencing rules for arm selection according to the current sample reward. The adaptive sequencing rules allow the algorithm to achieve a logarithmic regret order with time and to establish a finite sample bound for regret. This algorithm was demonstrated to be quite powerful (Adibi and van der Woude, 2022). proposed an RL-based solution for controlling the secondary frequencies of lossy-inverter-based microgrids. This solution does not require a priori information regarding system parameters and can achieve frequency synchronization within an ultimate bound in the presence of dominantly resistive impedances, inductive line and load impedances, model parameter uncertainties, and time-varying loads and disturbances. To develop a robust control strategy, Lyapunov theory was applied to establish a feedback control mechanism based on the unknown dynamics of the microgrid. Subsequently, a performance function based on the cumulative cost of convergence at the nominal frequency was approximated in real time by a critic network. An actor network simultaneously learned a parameterized approximation of the system's nonlinear dynamics while optimizing the performance function approximated by the critic network. Finally, the uniform ultimate boundedness of both the closed-loop frequency error dynamics and network weight estimation errors was demonstrated using the Lyapunov approach. Wen and Wang (2024) determined that integrating a Lyapunov controller with a conventional sliding-mode controller and a proportional–integral–derivative controller enables the stabilization of a nonlinear single-point levitation system in magnetic levitation trains. Moreover, Precup et al. (2021) noted that both control theory and data characteristics should be considered to ensure system stability within a data-driven framework. Accordingly, the proposed approach considers these two aspects.

The combination of RL with nature-inspired algorithms has inspired major advancements in control system design. For example, Zamfirache et al. (2025) proposed a method that combines the deep deterministic policy gradient (DDPG) algorithm with the slime mold algorithm (SMA). In this DDPG–SMA method, the SMA is used to initialize the parameters of a neural network controller, thus mitigating the instability that may arise from random initialization in conventional DDPG controllers. This method incorporates nature-inspired optimization to enforce state safety constraints; it is also consistent with the design of the neural network–based estimation controller proposed by Popova et al. (2003). Accordingly, the DDPG–SMA method and the neural network–based estimation control method demonstrate that the incorporation of artificial intelligence algorithms into complex control systems effectively enhances controller performance and stability.

In (Luipers et al., 2022), the authors used deep RL for planning actions and examining the relevant inverse kinematics. This hybrid approach was tested in three simulation tasks, and it outperformed a conventional approach that only involved RL (el Zaatari et al., 2021). used RL to optimize the paths generated by collaborative robots. In their RL process, they used the knowledge learned from demonstrated tasks as task parameters, which are critical inputs for generating a movement path for collaborative robots in a new dynamic task. This model is called Task-Parameterized Learning from Demonstrations. Such RL-based optimization can enable the identification and elimination of irrelevant parameters, thereby improving the overall computational performance (Yang et al., 2022). attempted to prevent the collisions caused by the direct grasping of tightly stacked objects and programmed a robotic arm to push the objects apart before grasping them individually, thereby increasing the success rate of grasping. A deep Q-learning framework was integrated with a non-maximum suppression policy to constrain unreasonable actions, thereby achieving the dynamic evaluation of pushing and grasping actions. This framework was also combined with a data-driven pushing reward network that could effectively assess the degree of separation or aggregation between objects. This approach was verified to be relatively successful in separating and grasping objects in real scenarios. The aforementioned literature review indicates that robotic arms with RL ability have been extensively used across various fields. Such robotic arms can also be used in other fields, such as visualization, big data analysis, and artificial intelligence. In this study, the RL of the positions of the end points of a robotic arm and the direction of its gripper was performed using different algorithms, and a reward mechanism for point tracking was designed using the distances between the end points and the target point as well as the angle of the gripper facing the target direction. The performance of the models trained using the different algorithms was then compared. Subsequently, environmental observations and a reward function were designed using the soft actor–critic (SAC) algorithm for obstacle avoidance (Tang et al., 2023; C. Wang et al., 2024). Compared with conventional kinematic calculations and control model, the proposed approach can control arms with higher accuracy and agility. The performance of the proposed model was verified by implementing it with a robotic arm to perform object picking tasks.

Zhang et al. (2023) proposed a hierarchical semantic knowledge–based object search model for household robots. They used Rao–Blackwellized particle filters and the improved rapidly exploring random tree algorithm for localization and path planning. They

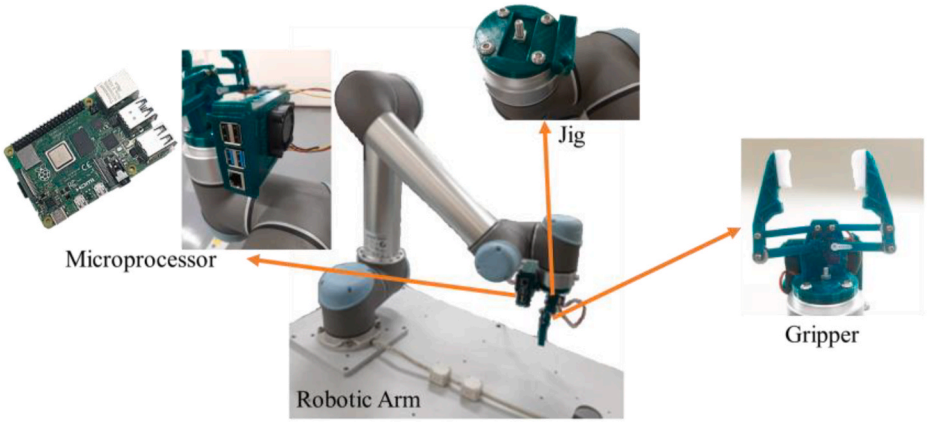


Fig. 1. UR5 robotic arm and its attachments.

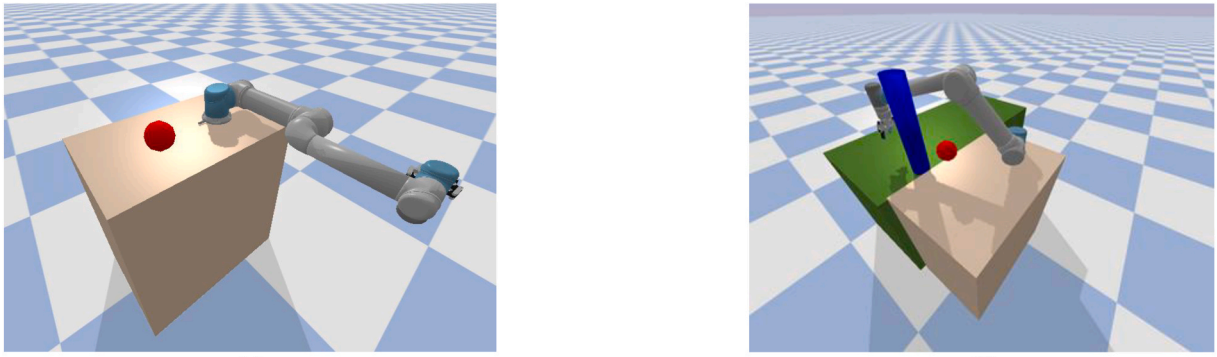


Fig. 2. Simulation environments for the (a) point tracking and (b) obstacle avoidance tasks.

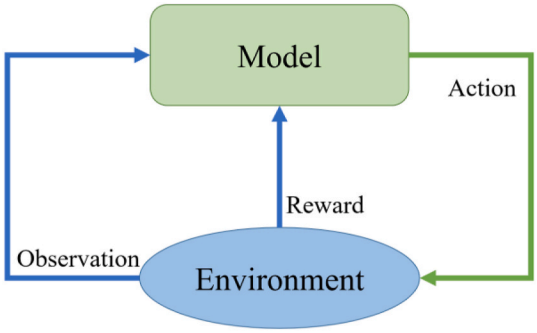


Fig. 3. Process of RL.

employed the developed object search model to control a robotic arm, revealing satisfactory performance. However, the model's applicability to general autonomous learning tasks is limited because their study primarily focused on the development of an inverse kinematics control model. Sun et al. (2022) developed a dual-arm mobile robot equipped with a visual system for various tasks. In their study, robotic arm control was mainly based on impedance control. The developed robot exhibited excellent performance but lacked an autonomous learning functionality; this thus limits its applicability to broader autonomous learning tasks. Moreover, Jeong et al. (2017) applied a model based on deep adaptive resonance theory to enable a robot to have autonomous learning capabilities and accomplish multiple tasks. Their experimental results indicated that the robot achieved excellent performance; however, they primarily used a conventional inverse kinematics model for robotic arm control, which may limit flexibility in complex or dynamic environments.

Accurately identifying obstacles is a critical requirement in the adaptive control of robotic arms for obstacle avoidance. Tao et al. (2024) developed the Enhanced Feature Extraction-You Only Look Once algorithm, which improves the detection of small objects in industrial environments (Hongfeng Tao et al., 2024). They also designed a control model based on state observation and input compensation, which exhibited excellent adaptability to changing environments (Tao et al., 2023). Du et al. (2024) employed fuzzy control to develop a control model for multiagent systems; this model can enable obstacle avoidance by and cooperation among multiple robotic arms.

According to the aforementioned literature review, studies have achieved promising performance in controlling the trajectory of robotic arm joints, and effective trajectory tracking has been achieved by combining RL-based path planning with traditional inverse kinematics. However, controlling only the end effector's position through inverse kinematics cannot guarantee obstacle avoidance for individual joints. Accordingly, the present study developed a model that utilizes RL to control the trajectory of each joint in a robotic arm; the proposed model leverages the superior performance of RL for robotic arm obstacle avoidance.

The SAC algorithm was employed in the primary model for RL-based robotic arm control. Learning was also performed using three other algorithms: the proximal policy optimization (PPO) (Boudlal et al., 2024), advantage actor–critic (A2C) (Mohamadi et al., 2024), and trust region policy optimization (TRPO) (Araújo et al., 2022) algorithms. The results obtained with the aforementioned four algorithms were compared, and the SAC algorithm was found to outperform the other algorithms. Therefore, an obstacle avoidance task was performed using the SAC algorithm. This paper contains five sections. Section I describes the

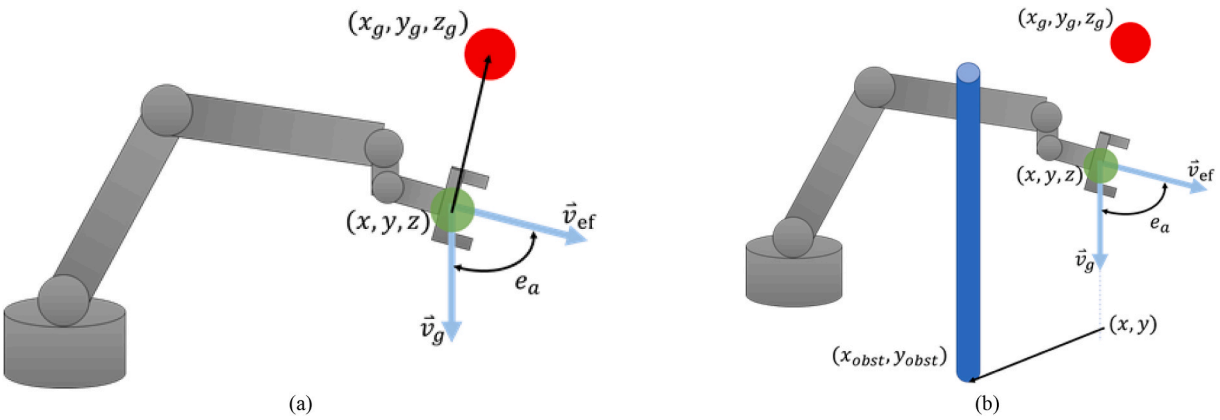


Fig. 4. Parametric schemas for the simulation environments: the (a) point tracking and (b) obstacle avoidance tasks.

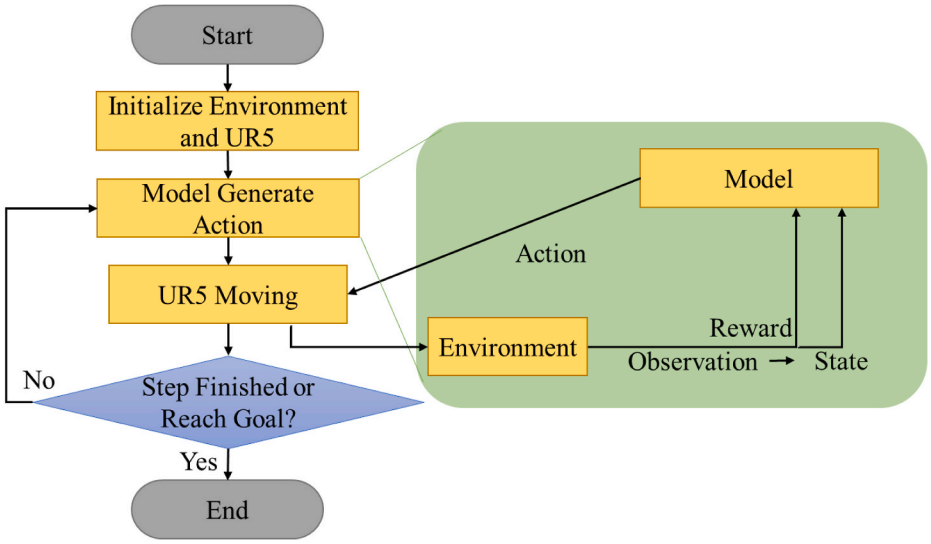


Fig. 5. Flowchart of the experimental process.

research background, the relevant literature, and the framework of this study. Section II details the system architecture, including the hardware, simulation environment, model inputs and outputs, and experiment process. Section III describes the controller design and RL, the principles and architectures of the adopted algorithms, the reward function, and the model performance indicators. Section IV presents the results obtained with the adopted models in simulated and actual point tracking and obstacle avoidance tasks. Section V provides the conclusions and contributions of this study.

Studies have demonstrated the application and effectiveness of various innovative methods in complex control systems. For example, Koprinkova-Hristova (2010) proposed a neuro-fuzzy controller training method based on backpropagation through time, which considerably improves the control accuracy and stability of nonlinear dynamic systems. Moreover, Roman et al. (2022) developed an iterative feedback tuning algorithm for controlling tower crane systems; the algorithm optimizes controller parameters to enhance the performance of a proportional–integral controller. On the basis of existing RL techniques, Zamfirache et al. (2024) developed an adaptive RL control method that integrates PPO with the SMA. They validated it through experiments on a tower crane system, and the results demonstrated its applicability and stability in highly dynamic environments. These studies provide a solid theoretical foundation for nonlinear system control, parameter optimization, and adaptive control in dynamic environments and have inspired further exploration of robotic arm control methods. Accordingly, the present study integrated the advanced techniques adopted in the aforementioned studies to develop the RL-based flexible control model for enhancing the performance of robotic arms in diverse application scenarios.

With the increasing demand for diversity and flexibility in manufacturing environments, scholars have explored adaptive control and path planning strategies to enhance smart manufacturing systems. Conventional fixed-path control methods struggle to accommodate frequently changing production requirements, particularly in dynamic environments, in which robotic arms must exhibit a high degree of self-adjustment and adaptability. For example, in human–robot collaboration, robotic arms must not only share workspaces with human operators but also dynamically adjust their paths in real time to avoid collisions. In addition, with the growing integration of multidisciplinary technologies, robotic arms are being increasingly deployed in medical assistance, warehouse logistics, and high-risk operational environments. These emerging applications necessitate the development of more flexible and efficient robotic arm control methods that can support the long-term goal of fully unmanned operation in smart factories and facilitate seamless human–robot collaboration and adaptation to dynamic environments. Accordingly, the present study leveraged RL to develop a model that can enable robotic arms to explore optimal paths and movements, thereby expanding their range of applications and improving their operational efficiency.

Studies have increasingly used various algorithms for robotic arm

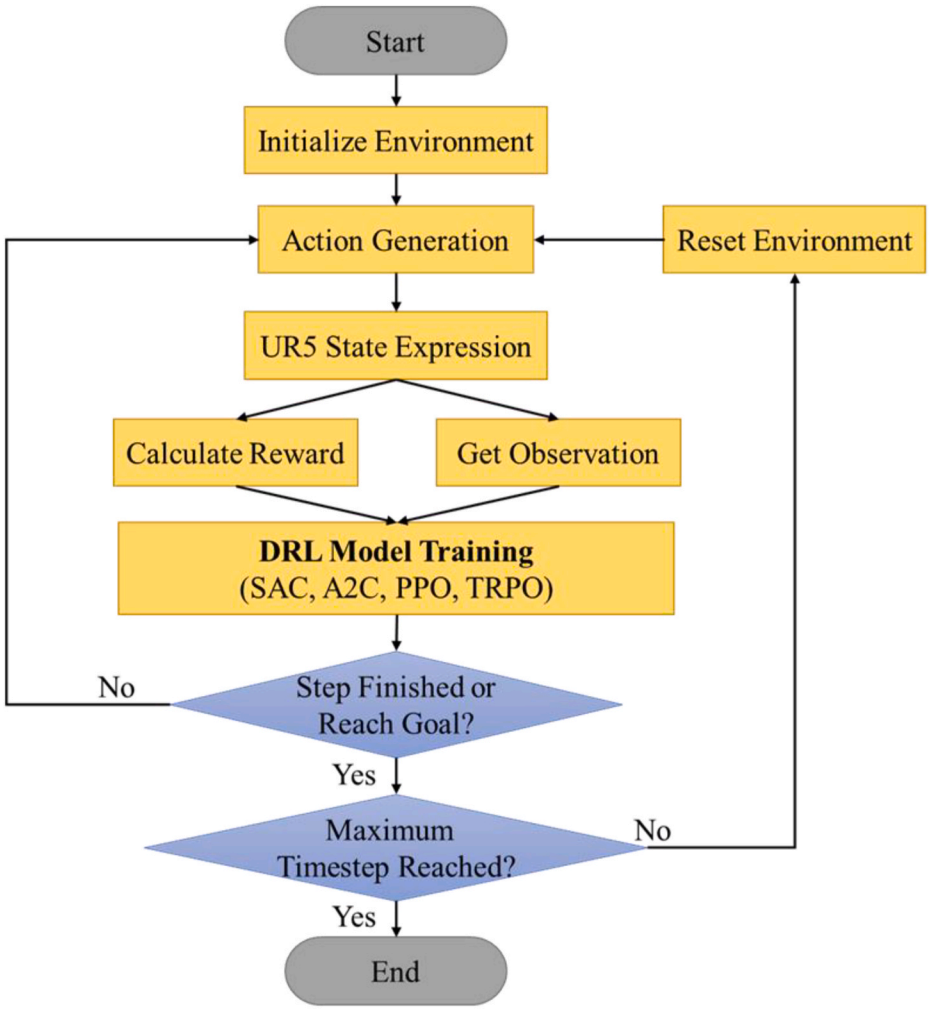


Fig. 6. Learning process of the designed robotic arm controller.

control, with multiple RL and planning methods being explored to enhance operational performance. Bing et al. (2022) introduced a graph-based distance metric to guide exploration. This approach is particularly suitable for environments with obstacles; however, it requires offline construction of an environmental map, and for robotic arms with several DoFs, state space may negatively affect learning efficiency. Furthermore, the aforementioned approach is primarily designed for object manipulation tasks, such as grasping and transportation, rather than dynamic obstacle avoidance and point tracking in systems with complex DoFs. Hu et al. (2023) proposed an inverse RL method to model biological adversarial behaviors and applied this method in dexterous hand grasping tasks. Although this method successfully reduced the contact force between the robotic arm and the object being grasped to improve the grasping success rate, it primarily focuses on enabling the robotic arm to adapt to the dynamic changes of living objects instead of facilitating point tracking and autonomous obstacle avoidance. Li et al. (2024) explored a hybrid approach that involves combining offline RL with a diffusion model. In this approach, trajectory generation and filtering are conducted to enhance policy stability. This approach has promise for pretraining robotic policies. However, because it primarily relies on offline data learning, it may face adaptability constraints in online control tasks that require continuous adaptation to environmental changes, such as point tracking and dynamic obstacle avoidance, which were explored in the present study. In contrast to the aforementioned studies, the present study employed a SAC RL model and enhanced the learning efficiency of a robotic arm through adaptive adjustment of a reward function. The proposed model offers several advantages. First, in contrast to offline RL methods, the proposed model enables robotic arms to adapt to environmental changes in real time and learn new motion strategies dynamically. Second, unlike graph-based hindsight goal generation, which primarily focuses on static target manipulation, the proposed model allows robotic arms to track target points while performing autonomous obstacle avoidance in complex environments. Finally, the entropy regularization mechanism incorporated into the SAC framework ensures that balance is achieved between exploration and exploitation, and the designed adaptive reward function can enhance learning efficiency. Accordingly, the proposed model is particularly suitable for controlling high-DoF robotic arms.

The literature indicates that conventional control methods without RL can provide stable and efficient performance in fixed or predictable tasks. However, these methods often exhibit limitations in complex and dynamic environments, which prevents them from addressing unforeseen situations or unstructured settings. The SAC-based model developed in this study exhibits theoretical robustness, and it leverages deep RL to enable autonomous learning of optimal strategies, thereby enhancing the intelligence and flexibility of robotic control. Specifically, this model facilitates adaptive control in dynamic environments without requiring manual parameter adjustments for different tasks. Moreover, it allows robotic arms to learn from mistakes and enables the continual optimization of their behavioral strategies for achieving highly efficient and flexible motion control. These advantages render the proposed model particularly suitable for applications requiring diversity and flexibility.

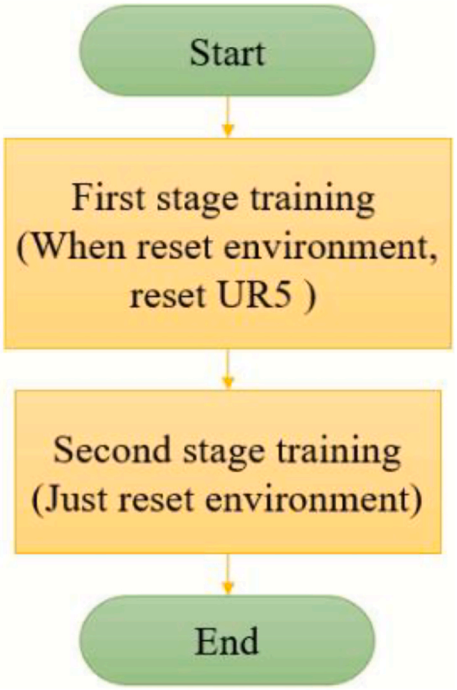


Fig. 7. Two-stage learning process.

2. System architecture

The UR5 robotic arm (Universal Robots) was used in this study. This robotic arm is a collaborative robot (cobot) with six DoFs and embedded joint modules; its appearance and the positions of its attachments are depicted in Fig. 1. The joints of UR5 have a rotation range of ±360°, which exceeds that of conventional robotic arms. A self-devised jig was used to connect a gripper to the robotic arm and secure the microprocessor of this arm. Picking actions performed by the gripper were driven by a servo motor controlled by the microprocessor.

The two principal tasks to be performed by the arm were point tracking and obstacle avoidance. The point tracking task mainly involved training a model to control the robotic arm to arrive at a designated position from any position. The obstacle avoidance task involved enabling the robotic arm to avoid an obstacle autonomously. The simulation environments for the point tracking and obstacle avoidance tasks are displayed in Fig. 2 (a) and 2 (b), respectively. Training and testing in simulation environments can drastically reduce the risks that might occur during physical testing, and successful simulations and physical testing can improve the feasibility of practical applications.

The model used to control the robotic arm was trained through RL according to the schema depicted in Fig. 3. Following each interaction between the model and the environment, the model was trained to fit the environment by learning from its mistakes. In the initial state, the environment output observations to describe the environment state. These observations and the reward value were input into the model, which then generated a set of best actions. After these actions were performed by the model, the modified environment output its new observations and reward to the model, which continued to learn iteratively. The reward was a feedback item based on the action produced by the model and had a critical influence on the learning results. The definition of the reward function is presented in Section III.

Model training requires inputs and outputs. Environmental observations are model inputs and convey information regarding the environment's current state to the model being trained. The considered observations vary according to the relevant task. In point tracking tasks, only observations about the robotic arm's status as well as the position and angular relations between its end point and the target point are required. Therefore, the environmental observations for the aforementioned tasks are represented as a matrix consisting of the angles of each joint of the robotic arm, the vector from the end point of the arm to the target point, and the angular difference between this end point and the target point. The parametric schema for point tracking tasks is depicted in Fig. 4(a). In contrast to point tracking tasks, obstacle avoidance tasks require information on not only the status of the robotic arm but also the position and angle of its end point. In addition, obstacle avoidance tasks require information regarding the positions of the robotic arm and obstacle. Therefore, for these tasks, the environmental observation matrix comprises the angles of each joint of the robotic arm, the position and angle of the end point of the arm, the target position, and the vector from the end point to the obstacle. The parametric schema for obstacle avoidance tasks is depicted in Fig. 4(b). When obstacle avoidance must be performed, the target position is tentatively adjusted to 0 so that this position is temporarily not considered by the model, thereby allowing the model to focus on obstacle avoidance.

The model output is the set of actions to be performed by the robotic arm. To increase the stability of the control model of the robotic arm, actions were output as variations in joint angles in this study.

Fig. 5 depicts a flowchart of the experimental process. After the environment and robotic arm posture are initialized, the trained model generates appropriate actions based on the current statuses of the robotic arm, thereby allowing the robotic arm to complete the required task. The robotic arm subsequently performs the movement corresponding to the action output. If the robotic arm is determined to have reached the preset maximum time step or target position, the experimental process is stopped.

3. The proposed model

RL was employed to train different models to control the adopted robotic arm autonomously by learning from its failures. Four models were trained and used in a point tracking task, and their performance was then compared. Subsequently, the model with the best point tracking performance was selected to perform obstacle avoidance. Because of the possibility of collisions during RL, which can damage equipment, training and testing were performed in simulated environments to minimize the probability of dangerous accidents occurring during physical testing. This section describes the training process, adopted training algorithms, reward function, and model evaluation indicators.

Fig. 6 depicts the learning process of the designed robotic arm controller. The robotic arm is initially set to have a random joint position; on the basis of this position, the first set of actions is generated. The robotic arm then moves according to the generated actions, and observations regarding the arm state and the reward resulting from the action are entered into different models for learning. Subsequently, whether the target point is reached or the preset time step is exceeded is determined. If not, the models continue to generate actions and repeat the aforementioned steps to continue learning. If yes, whether the maximum time step has been reached is determined. If the maximum timestep has not yet been reached, the environment is reset; this process involves initializing some parameters, changing the target point, and returning the arm to the starting position. After the reset, new actions are generated, and learning based on the resulting observations and rewards is performed. Conversely, if the maximum time step has been reached, the learning process is terminated, and the model is exported. The SAC, A2C, PPO, and TRPO algorithms were the RL algorithms used in this study, and all of these algorithms use multilayer perceptron policy networks.

The final point tracking model is obtained after at least two stages of learning (Fig. 7). During the first learning stage, when the environment is reset, the robotic arm is returned to its initial position to allow the model to learn the movements required in simple situations. During the second and subsequent learning stages, when the environment is reset,

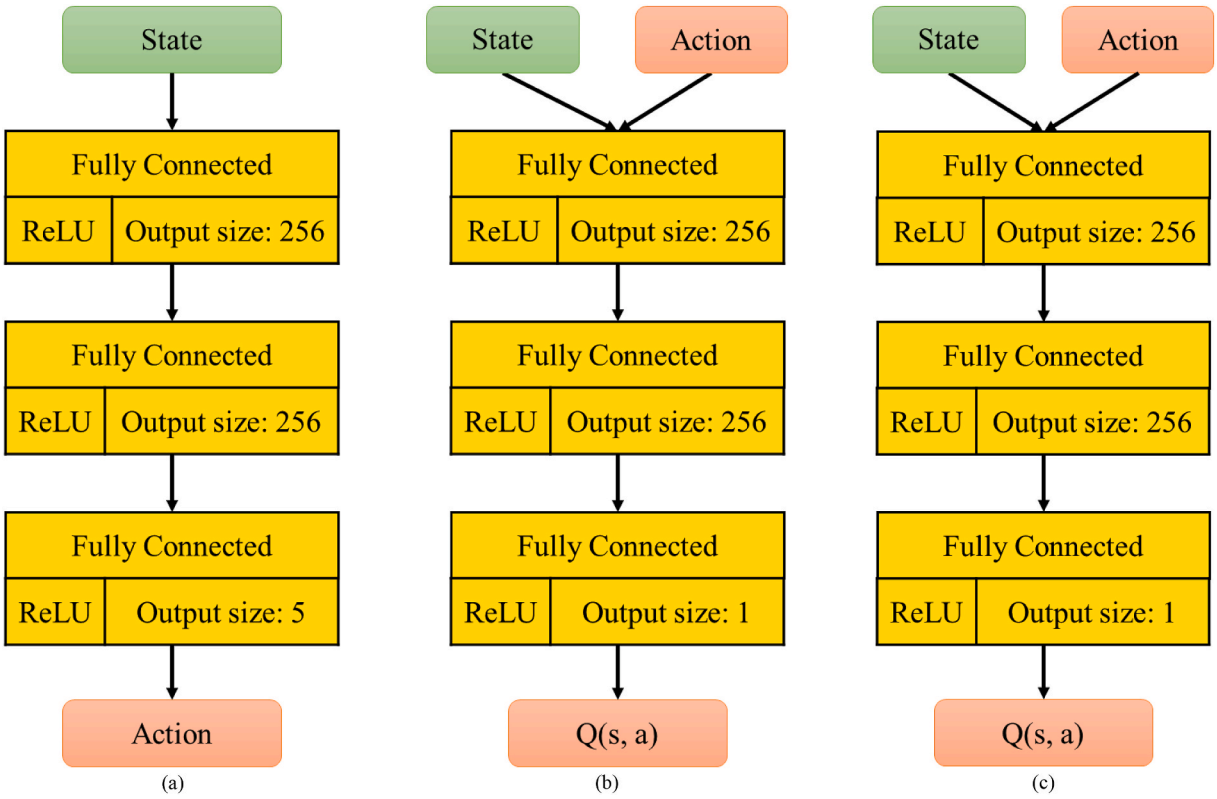


Fig. 8. SAC neural network architecture: (a) Action network. (b) Critic network. (c) Target network.

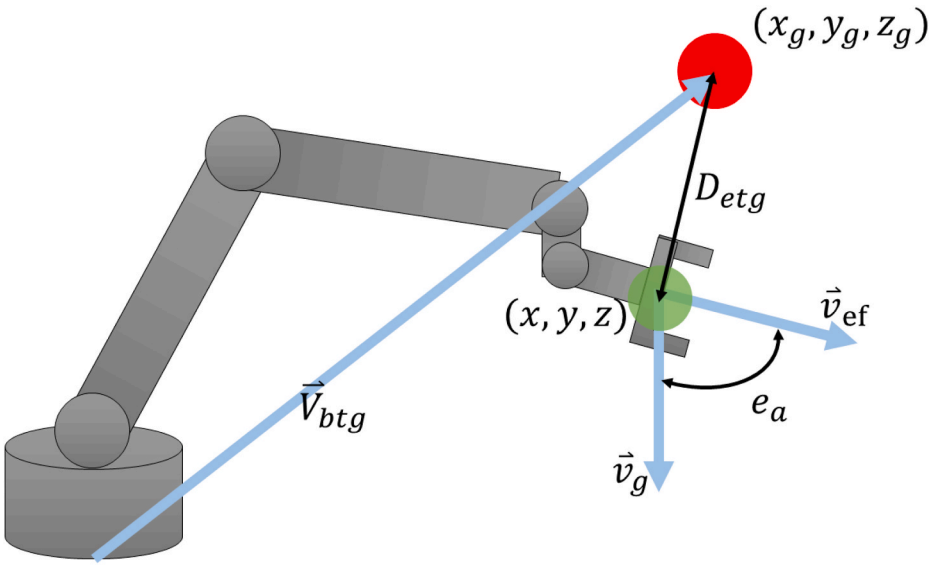


Fig. 9. Parameters of the reward function.

the arm is no longer returned to the original position because the model learns how to move the arm to the target position from the current position. This process is difficult, and the results of the first learning stage are used as a foundation to accelerate the subsequent learning stages.

3.1. TRPO algorithm

TRPO is an on-policy approach based on policy gradients. This approach was proposed by Schulman et al. (2015) to address the problem of low sampling efficiency in policy gradient models by performing importance sampling and to increase training stability by using a trust region (Schulman et al., 2015). The strategy parameters are updated using the following equation:

$$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{g^T H^{-1} g}} H^{-1} g \tag{1}$$

where α^j is the backtracking coefficient, which is obtained by searching or shrinking the trust region, and j is the smallest nonnegative integer that can improve the sample loss and ensure that the Kullback–Leibler (KL) divergence is less than δ.

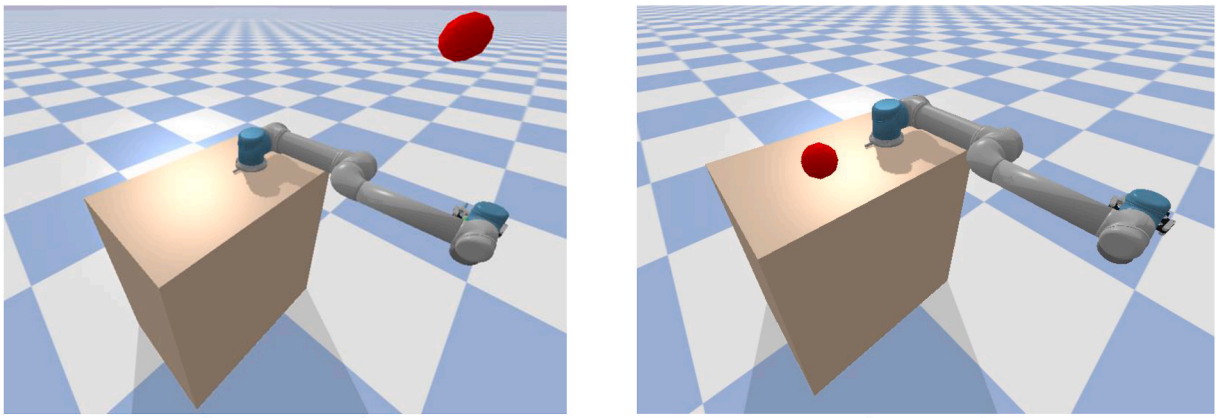


Fig. 10. Relative position of the test dot to the robotic arm in the simulated environment.

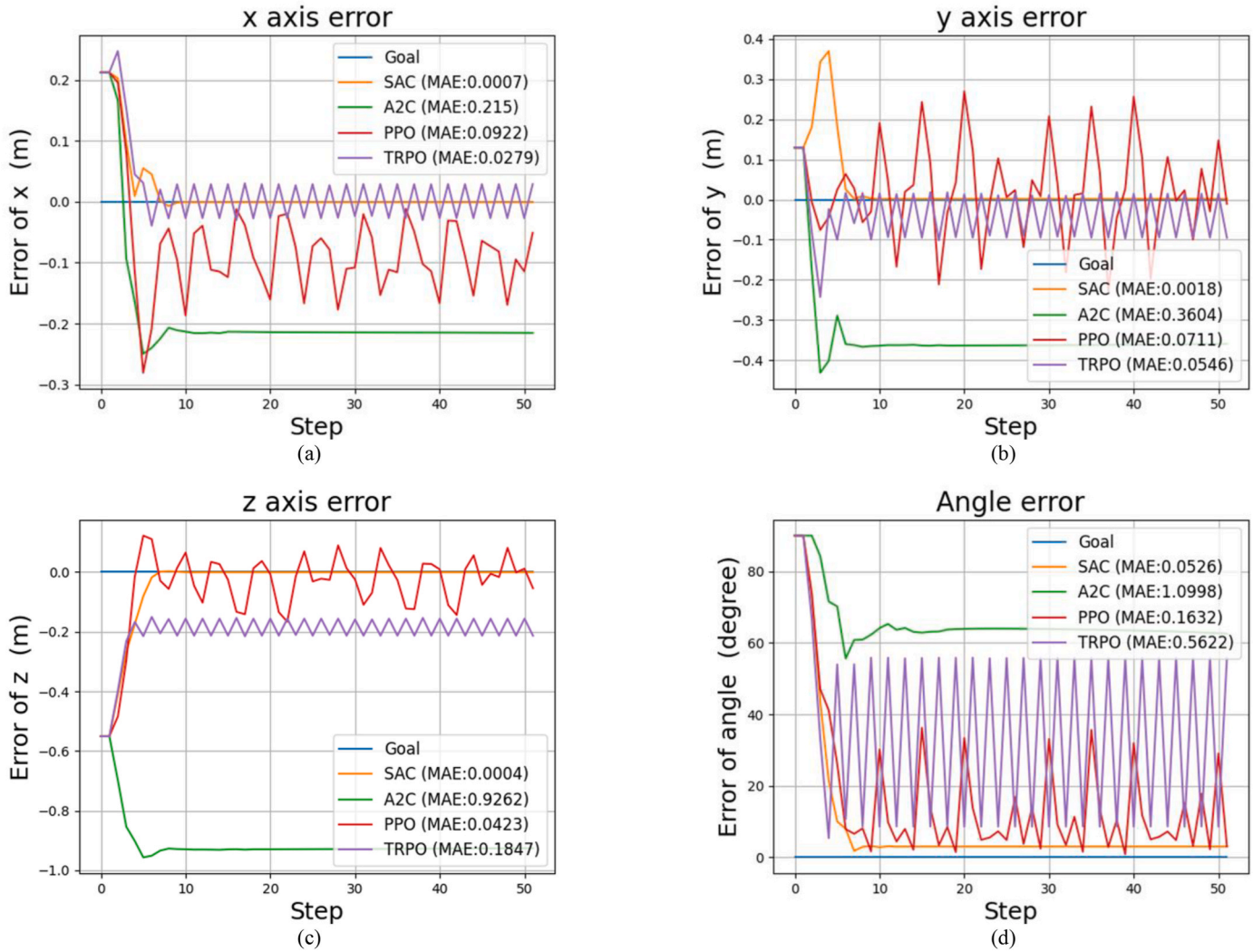


Fig. 11. Point tracking results for the first target: (a) x-axis error, (b) y-axis error, (c) z-axis error, and (d) angular error.

The policy gradient g is calculated using the expression $Hx = g$. Moreover, after the appropriate backtracking coefficient α^j is identified through a linear search, θ is updated to determine the optimal θ value for the strategy function. The pseudocode for the TRPO procedure, including the calculation of the policy gradient (g) and the backtracking search for α, is presented in Algorithm 1. The computational complexity of TRPO is $O(N^2)$ because of its trust region optimization process and natural gradient calculation, where N is the number of policy parameters.

Algorithm 1. Trust Region Policy Optimization (Joshua Achiam,

2020)

Input: Initial policy parameters θ_0 and initial value function parameters ϕ_0
Hyperparameters: KL divergence limit (δ), backtracking coefficient (α), and maximum number of backtracking steps (K)
for $k = 0, 1, 2, \ldots$ **do**
  Collect the set of trajectories $D_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
  Compute the rewards-to-go $\widehat{R}_t$.
  Compute the advantage estimate $\widehat{A}_t$ (by using any model of advantage estimation) on the basis of the current value function V_{ϕ_k}.
  Estimate policy gradient as $\widehat{g}_k = \frac{1}{|D_k|}\sum_{\tau\in D_k}\sum_a \nabla_\theta \log \pi_\theta(a_t|s_t)|_{\theta_k}\widehat{A}_t$
  Use the conjugate gradient algorithm to compute $\widehat{x}_k \approx \widehat{H}_k^{-1}\widehat{g}_k$ where $\widehat{H}_k$ is the Hessian of the sample-averaged KL divergence.
  Update the policy through backtracking line search with $\theta_{k+1} = \theta_k + \alpha^j\sqrt{\frac{2\delta}{\widehat{x}_k^T\widehat{H}_k\widehat{x}_k}}\widehat{x}_k$ where $j \in \{0, 1, 2, \ldots K\}$ is the smallest value that improves the sample loss and satisfies the KL divergence constraint for the sample.
  Fit the value function through regression on the mean square error: $\pi_{k+1} = \text{argmin}_\pi \frac{1}{|D_k|T}\sum_{\tau\in D_k}\sum_{t=0}^{T}(V_\pi(s_t) - \widehat{R}_t)^2$ typically by using some gradient descent algorithm.
end for

3.2. PPO algorithm

PPO is an on-policy approach that is an improvement over TRPO and was developed by Schulman et al. (2017). The main difference between PPO and TRPO is related to the use of constraints. The KL divergence and constraint conditions are not used in PPO; instead, a clipped objective is used to prevent the updated policy from changing excessively, thereby simplifying the solution. PPO updates are performed according to the following equation.

$$\theta_{k+1} = argmax_\theta E_{s,a\sim\pi_{\theta_k}}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a), clip\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)\right] \quad (2)$$

where ϵ is a hyperparameter that indicates the approximate difference before and after the policy update. PPO limits the importance weight to $(1-\epsilon, 1+\epsilon)$, thereby preventing excessive gradient changes. When the advantage function A is greater than 0, the action is beneficial to (s, a), and the probability of the action should be increased. However, if the advantage function exceeds $1+\epsilon$, the gradient increase would be excessively rapid.

The pseudocode for PPO is presented in Algorithm 2. This pseudocode indicates how the clipped importance weight is used to update the policy parameter θ in Equation (2). After the importance weight is limited, the updated strategy function parameter θ can be determined. PPO's computational complexity is $O(NK)$, where N is the number of network parameters and K is the number of epochs per update.

Algorithm 2. Proximal Policy Optimization-Clip (Joshua Achiam, 2020)

Input: Initial policy parameters θ_0 and initial value function parameters ϕ_0
for $k = 1, 2, \ldots$ **do**
Collect the set of trajectories $D_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
Compute the rewards-to-go $\widehat{R}_t$.
Compute the advantage estimate $\widehat{A}_t$ (by using any model of advantage estimation) on the basis of the current value function V_{ϕ_k}.

(continued on next column)

(continued)

Update the policy by maximizing the PPO-Clip objective: $\theta_{k+1} = argmax_\theta \frac{1}{|D_k|T}\sum_{\tau\in D_k}\sum_{t=0}^{T}\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a), g(\epsilon, A^{\pi_{\theta_k}}(s,a))\right)$ typically through stochastic gradient descent with the Adam optimizer.
Fit the value function through regression on the mean square error: $\phi_{k+1} = \text{argmin}_\phi \frac{1}{|D_k|T}\sum_{\tau\in D_k}\sum_{t=0}^{T}(V_\phi(s_t) - \widehat{R}_t)^2$ typically by using some gradient descent algorithm.
end for

3.3. A2C algorithm

A2C is an off-policy approach derived from actor–critic model and is the synchronous version of asynchronous advantage actor–critic, which was proposed by Mnih et al. (2016).

In the A2C approach (Konda and Tsitsiklis, 1999), the actor uses a policy-based model to control the actions to be performed, and the critic rates the adopted action according to a value-based system. The A2C update process is described in Algorithm 3. The computational complexity of A2C is $O(N)$ for both the actor and critic networks during forward propagation, where N is the number of network parameters.

3.4. SAC algorithm

SAC is an off-policy approach developed by Haarnoja et al. (2018a) as a successor to soft Q-learning in the twin-delayed DDPG algorithm. Instead of performing soft Q-learning, the SAC algorithm conducts double Q-learning. This algorithm uses an entropy regularizer, which maximizes the trade-off between the expected reward and entropy through training. Entropy is a measure of randomness in a strategy, and an increase in entropy accelerates exploration and learning and can effectively prevent a strategy from converging to a local optimal solution. SAC achieves efficient computation and has a computational complexity of $O(N_1 + N_2)$, where N_1 and N_2 represent the parameters of the policy and dual Q-networks, respectively. The additional entropy calculations result in an additional complexity of only O(1) per action.

Algorithm 3. Advantage Actor–Critic (Wang et al., 2018)

Assume parameter vectors θ and θ_v
Initialize step counter $t \leftarrow 1$
Initialize episode counter $E \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  $t_{start} = t$
  Get state s_t
  repeat
    Perform at according to policy $\pi(a_t|s_t;\theta)$
    Receive reward rt and new state st+1
  $t \leftarrow t+1$
  Until terminal st **or** t-tstart = = tmax
  $A = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t,\theta_v) & \text{for non-terminal } s_t \end{cases}$
  for $i \in \{t-1, \ldots, t_{start}\}$ ***do***
  $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt θ: $d\theta \leftarrow d\theta + \nabla_\theta \log \pi(a_i|s_i;\theta)(A - V(s_i;\theta)) + \beta_e \partial H(\pi(a_i|s_i;\theta))/\partial\theta$
    Accumulate gradients wrt θ_v: $d\theta_v \leftarrow d\theta_v + \beta_v(A - V(s_i;\theta_v))(\partial V(s_i;\theta_v))/\partial\theta_v)$
  end for
  Perform update of θ using $d\theta$ and of θ_v using $d\theta_v$

(continued on next page)

(*continued*)

20. $E \leftarrow E + 1$
21. **until** $E > E_{max}$

The soft Q-value function is trained using the minimum soft Bellman residual, as expressed in (3), and the relevant gradient is expressed in (4).

$$J_Q(\phi) = E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\left(Q_\phi(s_t,a_t) - \left(r(s_t,a_t) + \gamma E_{s_{t+1}\sim\rho\pi(s)}[V_{\bar{\psi}}(s_{t+1})]\right)\right)^2\right] \quad (3)$$

$$\nabla_\phi J_Q(\phi) = \nabla_\phi Q_\phi(s_t,a_t)\left(Q_\phi(s_t,a_t) - r(s_t,a_t) - \gamma V_{\bar{\psi}}(s_{t+1})\right) \quad (4)$$

Strategy network parameters are updated using the minimized KL divergence, and the constant $\log Z(s_t)$ value that does not affect the gradient is removed, as expressed in (5); the relevant gradient is expressed in (6).

$$\begin{aligned} J_\pi(\theta) &= D_{KL}\left(\pi_\theta(.|s_t) \parallel exp\left(Q_\phi(s_t,.) - log\, Z(s_t)\right)\right) \\ &= E_{s_t\sim D, a_t\sim\pi_\theta}\left[log\left(\frac{\pi_\theta(a_t|s_t)}{exp\left(Q_\phi(s_t,a_t) - log\, Z(s_t)\right)}\right)\right] \\ &= E_{s_t\sim D, a_t\sim\pi_\theta}\left[log\,\pi_\theta(a_t|s_t) - Q_\phi(s_t,a_t) + log\, Z(s_t)\right] \end{aligned} \quad (5)$$

$$= E_{s_t\sim D, a_t\sim\pi_\theta}\left[log\,\pi_\theta(a_t|s_t) - Q_\phi(s_t,a_t)\right] \nabla_\theta J_\pi(\theta) = \nabla_\theta\, log\,\pi_\theta(a_t|s_t)$$

$$+ \left(\nabla_\theta\, log\,\pi_\theta(a_t|s_t) - \nabla_\theta Q_\phi(s_t,a_t)\right)\nabla_\theta a_t \quad (6)$$

Updated target parameters can be calculated using the aforementioned update formula, and the update process is described in Algorithm 4. In Equations (5) and (6), θ is the parameter of the update strategy, and ϕ is the updated Q-value. The SAC neural network architecture consists of an action network, a critic network, and a target network, as depicted in Fig. 8.

In summary, the SAC algorithm achieves a balance between exploration and exploitation on the basis of the maximum entropy principle. The offline update mechanism of SAC enables it to store interaction data in a replay buffer, allowing for the repeated use of past experiences, which considerably enhances data utilization efficiency. Moreover, the automatic temperature adjustment mechanism of SAC dynamically tunes the temperature parameter (α) in response to environmental demands, ensuring an optimal trade-off between exploration and exploitation while preventing excessive exploration or overexploitation of the policy. SAC employs a double-Q-network architecture, in which two networks independently estimate the Q-value, and the policy update is based on the smaller of the two estimates. This design effectively mitigates the overestimation of the Q-value, thereby leading to a more stable learning process and improved data utilization efficiency (Haarnoja et al., 2018b).

3.5. Comparison of SAC, DDPG, and TD3

SAC, DDPG, and Twin Delayed Deep Deterministic Policy Gradient (TD3) are actor–critic-based RL algorithms. However, they differ in terms of their policy learning, stability, and exploration mechanisms. DDPG employs a deterministic policy, which restricts exploration during training and increases the risk of the algorithm becoming trapped in local optima. Although target networks can be used to stabilize learning, Q-value overestimation can still occur, potentially leading to unstable training. SAC builds upon DDPG by incorporating entropy regularization, which encourages the policy to maintain a certain level of randomness, thereby enhancing exploration capability and improving policy robustness. The experimental results of this study revealed that SAC-trained policies adapted effectively to different environments and exhibited stable behavioral patterns. TD3 improves upon DDPG by addressing the problem of Q-value overestimation. The TD3 algorithm employs two Q-networks to reduce estimation bias and uses delayed policy updates to enhance training stability. Although SAC also focuses on stabilizing Q-value estimation, it places greater emphasis on exploration and policy generalization than on ensuring consistent performance across various environments. Overall, each of the aforementioned methods has its own advantages. The proposed model exhibited stability and generalization capabilities comparable to those of SAC while also incorporating the learning efficiency and Q-value estimation advantages of DDPG and TD3. Thus, the proposed model enables more effective adaptation to common robotic arm control tasks than do SAC, DDPG, and TD3.

3.6. Reward function

The reward value affects the effectiveness of RL, and the training process performed in this study was based on two statuses of the robotic arm: its position and the direction of its end point. The parameters used to calculate the reward function are displayed in Fig. 9, in which the red dot represents the position of the target point, and the green dot represents the position of the end point of the mechanical arm.

Algorithm 4. Soft Actor–Critic (Haarnoja et al., 2018c)

Initialize parameter vectors θ, ϕ_1, ϕ_2
for each iteration **do**
  for each environment step **do**
    $a_t \sim \pi_\theta(a_t|s_t)$
    $s_{t+1} \sim p(s_{t+1}|s_t,a_t)$
    $D \leftarrow D \cup \{(s_t,a_t,r(s_t,a_t),s_{t+1})\}$
  end for
  for each gradient step **do**
    $\phi_i \leftarrow \phi_i - \lambda_Q \hat{\nabla}_\phi J_Q(\phi)$ for i ∈ {1, 2}
    $\theta \leftarrow \theta - \lambda_\pi \hat{\nabla}_\theta J_\pi(\theta)$
    $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$
    $\overline{\phi_i} \leftarrow \tau\phi_i + (1-\tau)\overline{\phi_i}$ for i ∈ {1, 2}
  end for
end for

As expressed in (7), the basic reward R_o is equal to the sum of the rewards based on distance and angle (R_d and R_a, respectively). The term R_d is calculated using (8), in which D_{etg} is the distance from the end point of the arm to the target, and $\vec{V}_{btg}$ is the vector from the base of the arm to the target. The term D_{etg} is calculated using (9), in which x, y, and z representing the coordinates of the end point of the robotic arm, and x_g, y_g, and z_g denote the coordinates of the target point. The term R_a is calculated using (10), in which e_a is the included angle between the direction of the end point of the robotic arm and the direction of the target point. The term e_a is calculated using (11), in which $\vec{v}_{ef}$ is the vector of the end point, and $\vec{v}_g$ is the vector from the end point to the target point.

The parameter R_d can be considered a score for the current position of the robotic arm; the farther is the arm from the target, the lower is the

Table 1
X-AXIS errors for the first target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0007**	0.215	0.0922	0.0279
RMSE	**0.0007**	0.215	0.101	0.0279
SD	**0**	0.0002	0.0411	0.0279

Table 2
Y-AXIS errors for the first target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0018**	0.3604	0.0711	0.0546
RMSE	**0.0018**	0.3604	0.0946	0.0678
SD	**0**	0.0006	0.0946	0.0546

Table 3
Z-AXIS errors for the first target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0004**	0.9262	0.0423	0.1847
RMSE	**0.0004**	0.9262	0.0598	0.1869
SD	**0**	0.0004	0.0589	0.0285

Table 4
Angular errors for the first target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0526**	1.0998	0.1632	0.5622
RMSE	**0.0526**	1.0998	0.2176	0.6968
SD	**0**	0.0042	0.1439	0.4117

Table 5
Positional error for the first target point after convergence (m).

	SAC	A2C	PPO	TRPO
x	**−0.0007**	−0.2150	−0.0922	0.0011
y	**0.0018**	−0.3604	−0.0005	−0.0402
z	**−0.0004**	−0.9262	−0.0107	−0.1847
Distance error	**0.0020**	1.0169	0.0928	0.1891

score. The same principle applies to R_a, with higher scores indicating smaller included angles.

$$R_o = R_d + R_a \tag{7}$$

$$R_d = -\frac{D_{etg}}{\left\|\vec{V}_{btg}\right\|} \tag{8}$$

$$D_{etg} = \sqrt{(x-x_g)^2 + \left(y-y_g\right)^2 + (z-z_g)^2} \tag{9}$$

$$R_a = -\frac{e_a}{180} \tag{10}$$

$$e_a = \cos^{-1}\left(\frac{\vec{v}_{ef}\cdot\vec{v}_g}{\left|\vec{v}_{ef}\right| \times \left|\vec{v}_g\right|}\right) \tag{11}$$

The final reward function in point tracking is expressed in (12). To prevent the robotic arm from moving back and forth at the same location during RL, the following condition is added: if the difference between the previous distance and the current distance is less than the precision requirement, points are deducted from the final reward value $R_{f,a}$ (13).

$$R_{f,a} = \begin{cases} R_o + r_s, & \textit{if distance} < e_{ad1} \textit{ and angle} < e_{aa} \\ R_o + r_{pf}, & \textit{if angle} < e_{aa} \textit{ and distance} < e_{ad2} \\ R_o + r_{af}, & \textit{if distance} < e_{ad1} \\ R_o, & \textit{if distance} > e_{ad1} \textit{ and angle} > e_{aa} \end{cases} \tag{12}$$

$$R_{f,a} = R_{f,a} - r_{Ds}, \text{When } D_t - D_{t-1} \ll d_{acc} \tag{13}$$

where $R_{f,a}$ is the ultimate reward value in point tracking, and r_s, r_{pf}, and r_{af} are the rewards assigned under three conditions. The conditional judgment [e_{ad1}, e_{aa}] depends on the distance tolerance and angular tolerance; e_{ad2} is the second distance tolerance and is marginally higher than e_{ad1}. Furthermore, r_{Ds} is the penalty, D_t is the current distance, D_{t-1} is the previous distance, and d_{acc} is the judgment value that determines whether the displacement is close to 0.

The ultimate reward function depends on whether obstacle avoidance is required. When the distance between the end point of the robotic arm and an obstacle is shorter than a certain threshold and the obstacle is between the end point and the target, obstacle avoidance is essential. The reward function for this scenario is expressed in (14). The ultimate reward for obstacle avoidance ($R_{f,b}$) is set as a certain value ($-r_{obst}$). To allow the model to focus on obstacle avoidance, the basic reward value R_o is temporarily ignored. If the robotic arm collides with an obstacle, points (r_{col}) are deducted from the set $R_{f,b}$ value ($-r_{obst} - r_{col}$) to train to model to avoid a repetition of this situation.

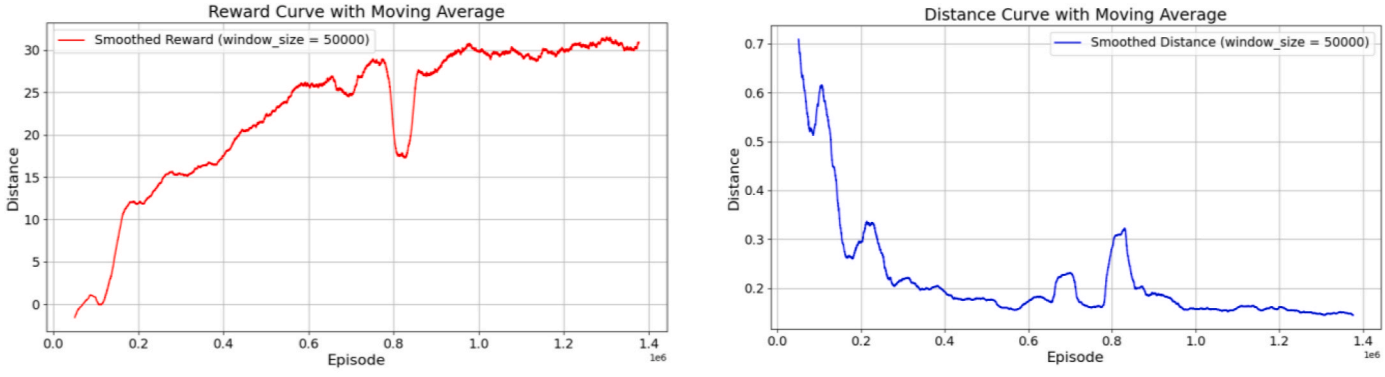
$$R_{f,b} = \begin{cases} -r_{obst}, \text{when avoid obstacle} \\ -r_{obst} - r_{col}, \text{when collide with obstacle} \end{cases} \tag{14}$$

where $R_{f,b}$ is the ultimate reward, r_{obst} is the penalty assigned when obstacle avoidance is deemed necessary, and r_{col} is the penalty assigned after a collision.

If obstacle avoidance is not required, the reward function for point tracking [(12) and (13)] is used with two additional conditions, as expressed in (15). The first condition is that although obstacle avoidance is not required, collision with an obstacle might still occur during movement, and if such a collision occurs, r_{col} shall be deducted from $R_{f,b}$ to minimize the repetition of such actions. The second condition is if the difference between the current distance and the distance before the previous one is smaller than the difference between the current distance and the previous distance, a small penalty is assigned to prevent the end point of the robotic arm from exhibiting reciprocal movements during training.

$$R_{f,b} = \begin{cases} R_{f,b} - r_{col}, & \text{when collide with obstacle} \\ R_{f,b} - r_{rec}, & \text{when } |D_t - D_{t-2}| < |D_t - D_{t-1}| \end{cases} \tag{15}$$

where r_{col} is the penalty assigned after a collision, r_{rec} is the penalty



(a) (b)

Fig. 12. Convergence curves obtained for reward and distance by using the SAC model.

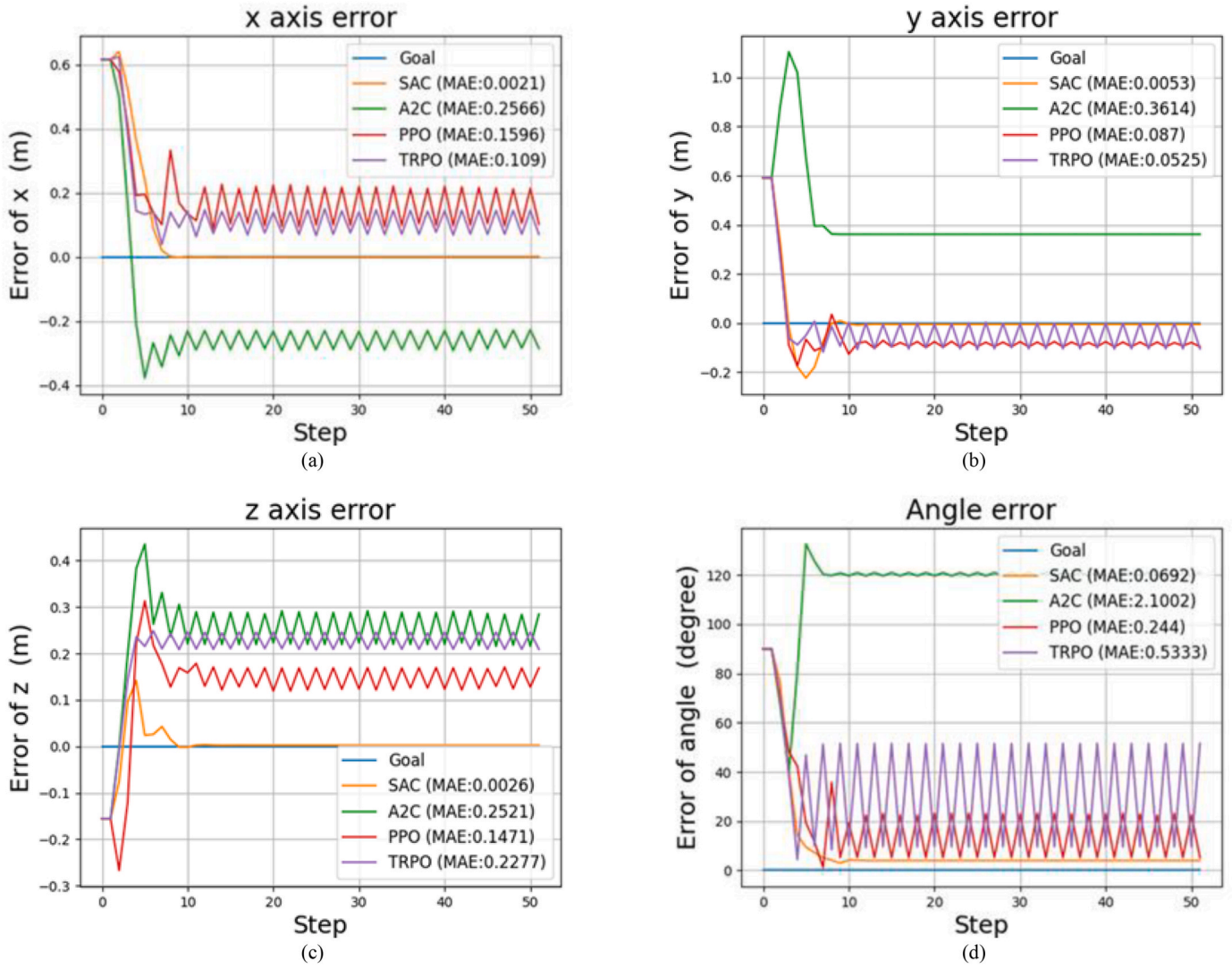


Fig. 13. Point tracking results for the first target: (a) *x*-axis error, (b) *y*-axis error, (c) *z*-axis error, and (d) angular error.

assigned when reciprocal movements occur, D_t is the current distance, D_{t-1} is the previous distance, and D_{t-2} is the distance before the previous distance.

Overestimation and underestimation biases in the actor–critic architecture can negatively affect policy stability and convergence. To address this problem, several optimization strategies were used in the design of the reward function to prevent the agent from becoming overly sensitive to specific actions. Moreover, a weighted loss function was incorporated into the critic update process to mitigate the risk of underestimation during training. The combined application of the aforementioned strategies enhanced model stability and performance while considerably reducing the influence of bias on the final results.

3.7. Model evaluation indicators

In this study, model performance was evaluated in terms of mean absolute error (MAE), root mean square error (RMSE), and standard deviation (SD).

MAE represents the mean absolute difference between the actual and predicted values and is calculated as follows:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|f_i - y_i| \tag{16}$$

where f_i is the estimated value and y_i is the actual value. Positive and negative errors might offset each other, which leads to unsatisfactory judgments; the use of MAE prevents this problem.

RMSE indicates the mean difference between the estimated and actual values and is expressed as follows:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{17}$$

Table 6
X-AXIS errors for the second target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0021**	0.2566	0.1596	0.109
RMSE	**0.0021**	0.2582	0.1695	0.1153
SD	**0**	0.0292	0.0571	0.0373

Table 7
Y-AXIS errors for the second target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0053**	0.3614	0.087	0.0525
RMSE	**0.0053**	0.3614	0.0874	0.0741
SD	**0**	0	0.0083	0.0525

Table 8
Z-AXIS errors for the second target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0026**	0.2521	0.1471	0.2277
RMSE	**0.0026**	0.2545	0.1487	0.2284
SD	**0**	0.0344	0.022	0.0185

Table 9
Angular errors for the second target point.

	SAC	A2C	PPO	TRPO
MAE	**0.0692**	2.1002	0.244	0.5333
RMSE	**0.0692**	2.1003	0.2886	0.6469
SD	**0**	0.0117	0.1542	0.3661

Table 10
Positional error for the second target point after convergence (M).

	SAC	A2C	PPO	TRPO
x	**0.0021**	−0.2566	0.1596	0.1090
y	**−0.0053**	0.3614	−0.0870	−0.0524
z	**0.0026**	0.2521	0.1471	0.2277
Distance error	**0.0063**	0.5099	0.2338	0.2578

where $\widehat{y_i}$ is the estimated value and y_i is the actual value. The smaller is the RMSE, the higher is the precision of the model.

SD describes the deviance of values from the mean and is expressed as follows:

$$SD = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \overline{x})^2}{n}} \tag{18}$$

where x_i is the value and $\overline{x}$ is the mean. The smaller is the SD, the more closely are the values clustered around the mean. If the SD is close to 0, the values are close to the mean. In addition to the aforementioned three indicators, angular error was considered. Angular error is calculated using (11).

4. Simulation and experimental results

In this study, a robotic arm was trained to perform point tracking by conducting RL with the SAC, PPO, A2C, and TRPO algorithms. Furthermore, the robotic arm was trained to perform obstacle avoidance by conducting RL using the SAC algorithm with a modified reward function. This section primarily describes the simulation and physical testing performed to verify the feasibility of the proposed model in practical scenarios. The experimental results can be accessed in (Chen-Ting Huang et al., 2023).

4.1. Model evaluation in simulated environments

To avoid accidents in practical scenarios, model validation was performed in a simulated environment. In this study, point tracking models based on the aforementioned four algorithms and an obstacle avoidance model based on the SAC algorithm were tested. The simulation results were recorded and analyzed. To evaluate our model's generalization capability, we randomly selected training and testing positions.

The point tracking results obtained with the four RL models were compared. The effectiveness of these models was evaluated according to their performance in tracking the dot depicted in Fig. 10. The errors in the coordinates of the robotic arm and the error in the angle between the end point of the robotic arm and the target point were determined for each model.

Figs. 11 and 13 display the model validation results obtained in the simulated environment. These figures indicate that the errors of the *x*–*y*–*z*-axes and angles obtained using the SAC model exhibited rapid convergence. For fairness, the models were validated using the final convergence results produced by them after a fixed number of steps. The results indicated that the SAC model remained stable after the error in the target point fell within the permissible range and steadily converged to the target point.

Tables 1, 2, 3, and 4 present the MAE, RMSE, and SD values of the *x*-axis, *y*-axis, *z*-axis, and angles, respectively, generated by the four adopted models. The last 10 numerical values were considered to calculate the MAE, RMSE, and SD because the first 10 values were used for position adjustment and had little referential significance. Table 5 presents the *x*-axis, *y*-axis, and *z*-axis errors of the models after their convergence. Final convergence was the primary basis for evaluating model performance; therefore, the last 10 numerical values were used in the distance formula. The results indicated that the SAC model outperformed the other models in terms of the *x*-axis, *y*-axis, *z*-axis, and angular errors.

The MAE and RMSE values indicated that the SAC model was stable after convergence. Furthermore, the SD values obtained with the SAC model for the aforementioned four parameters were 0, signifying that the model achieved convergence stability. Fig. 12 illustrates the convergence curves obtained for reward and distance by using the SAC model after the application of a moving average process. Following this

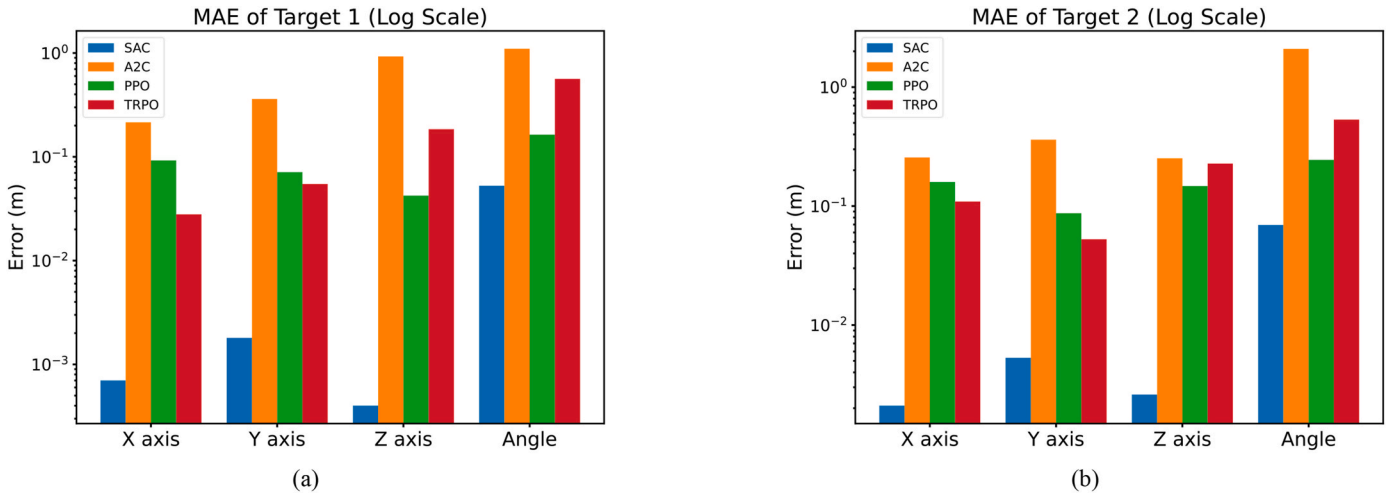


Fig. 14. MAE values of the four adopted models in the tracking process: (a) first target and (b) second target.

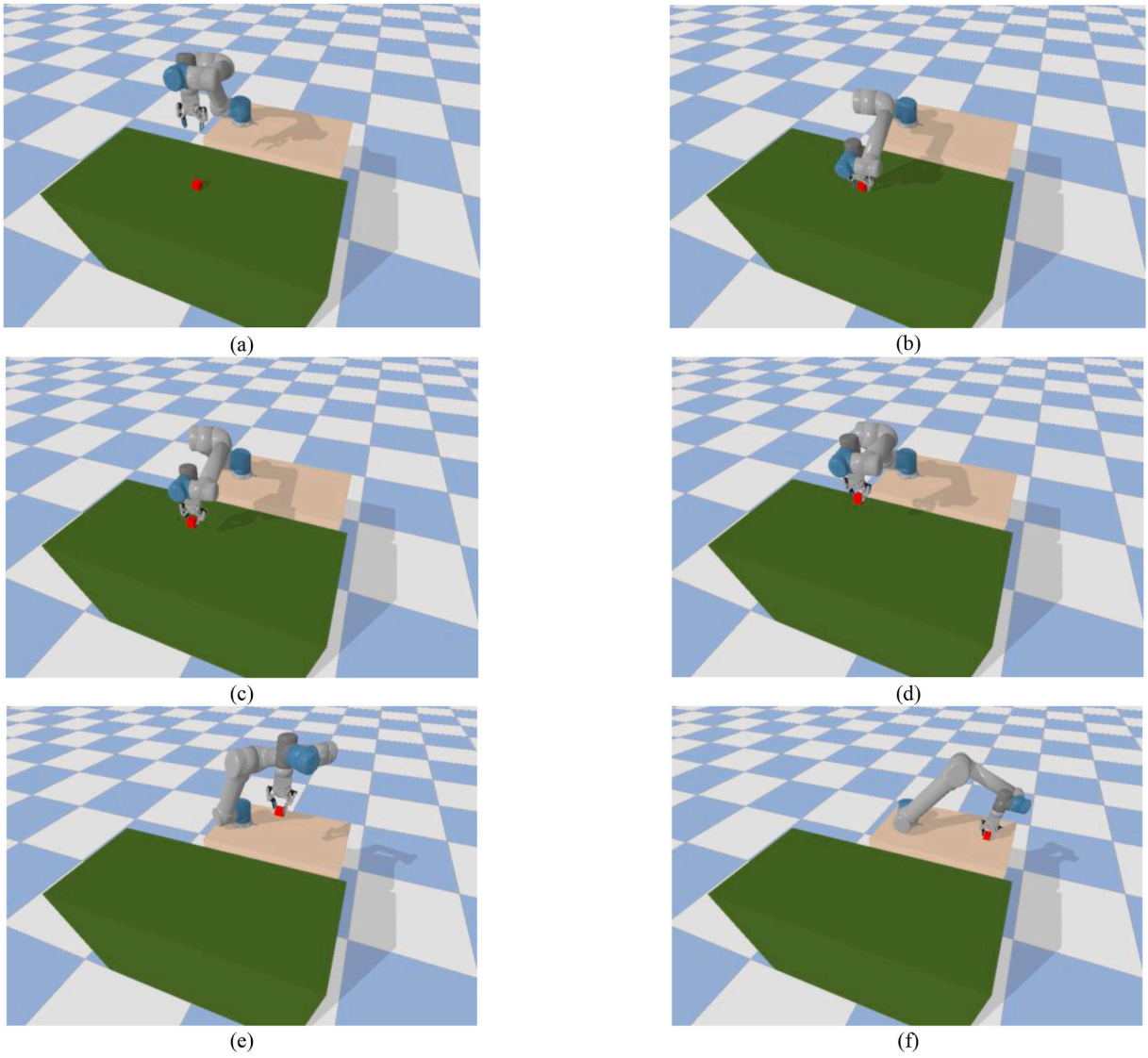


Fig. 15. Point-to-point testing of the SAC model in a simulated environment.

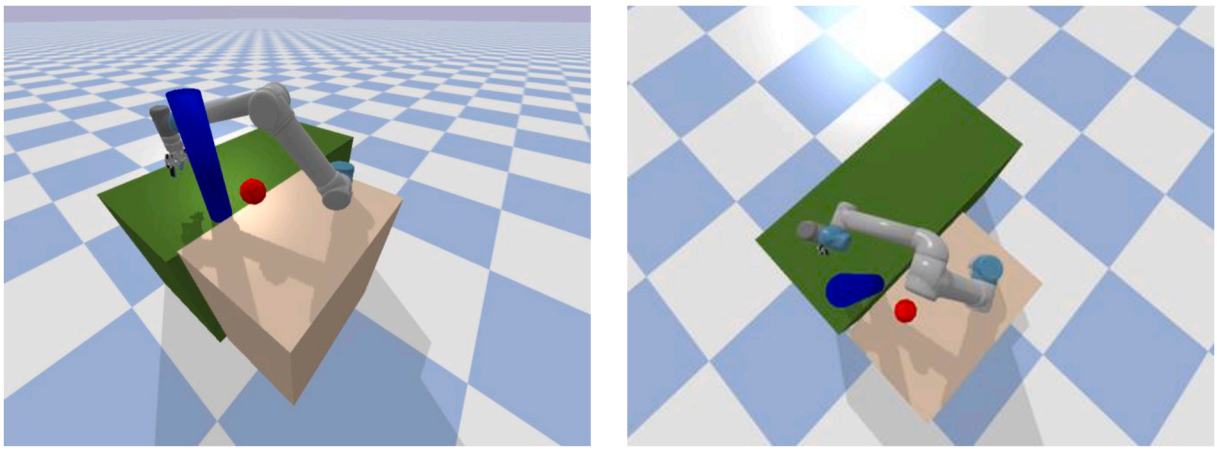


Fig. 16. Relative positions of the obstacle and UR5 robotic arm in the simulated environment (left is the typical view and right is the aerial view).

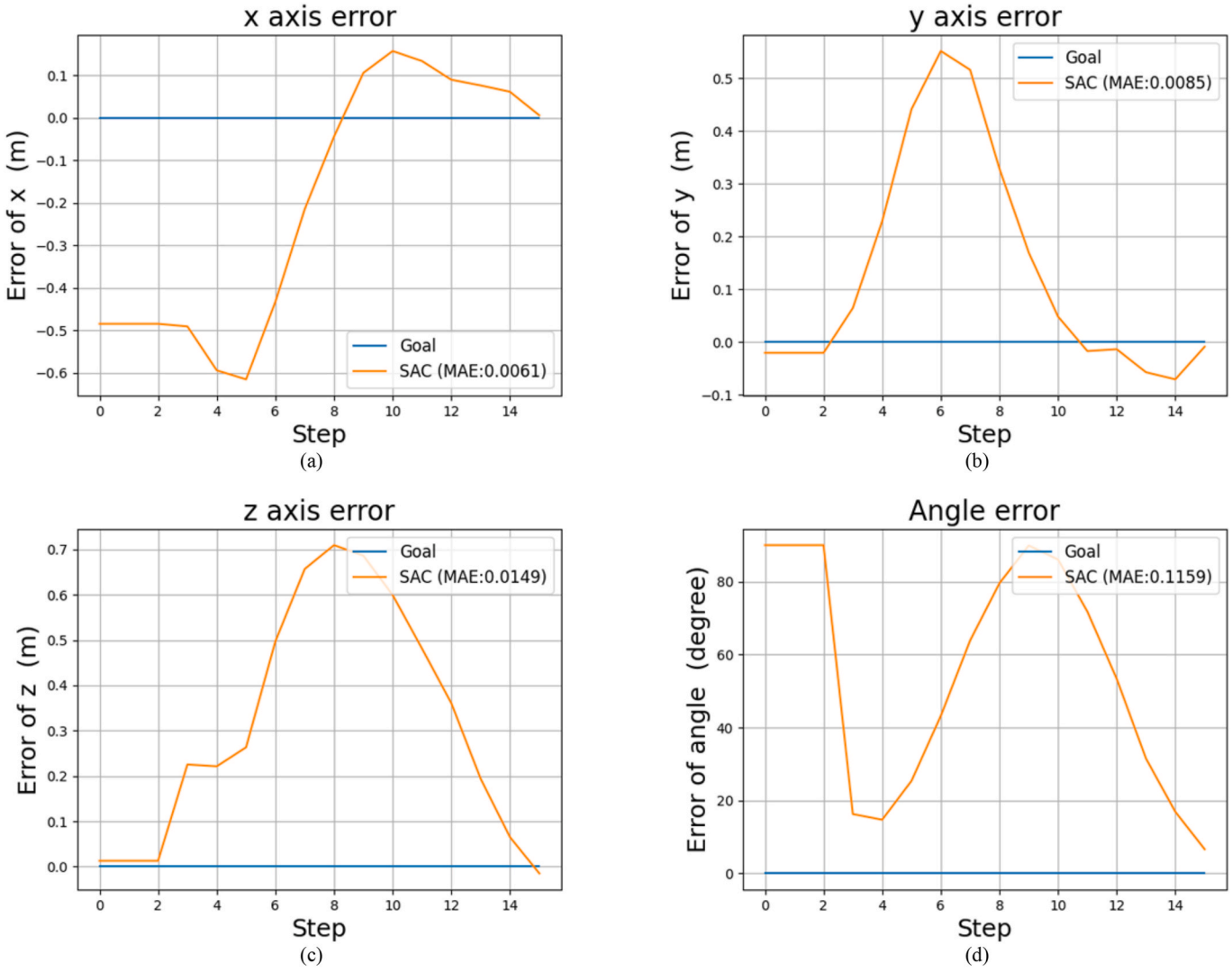


Fig. 17. Results obtained in the obstacle avoidance task: (a) *x*-axis error, (b) *y*-axis error, (c) *z*-axis error, and (d) angular error.

step, the reward exhibited a steady upward trend, and the distance to the target point gradually decreased. These results confirm that the designed reward function and the hyperparameter adjustments made during training had positive effects on model convergence, considerably enhancing learning stability and overall model performance.

The model validation results obtained for the second target position are shown in Fig. 13. Again, the SAC model outperformed the other models. Table6 and 7, 8, and Table9 present the MAE, RMSE, and SD values of the *x*-axis, *y*-axis, *z*-axis, and angle, respectively, generated by the four models. As presented in Table 10, compared with the PPO,

Table 11
Results obtained in the obstacle avoidance task.

	x-axis	*y*-axis	*z*-axis	Angle
MAE	0.0061	0.0085	0.0149	0.1159
RMSE	0.0061	0.0085	0.0149	0.1159
SD	0	0	0	0

Table 12
Final position of the end point relative to the target point during the obstacle avoidance task (m).

	x-axis	*y*-axis	*z*-axis	Distance error
SAC	0.006119	−0.00849	−0.0149	0.0182

TRPO, and A2C models, the SAC model exhibited not only faster convergence but also more stable performance. Fig. 14 depicts the MAE values of the four adopted models in the tracking process; these values indicated that the SAC model had the smallest error.

After model deployment in the simulated environment, the trained SAC model exhibited higher stability during object manipulation tasks than did the other three models (Fig. 15). In typical robotic arm operating environments, the SAC model effectively adapted to different scenarios, exhibited reduced trajectory deviations, and demonstrated improved operational precision and stability. These results indicate that the SAC model had strong generalization capabilities, which enhanced the efficiency and reliability of robotic arm operations.

In the obstacle avoidance task, the SAC model was combined with different reward functions, enabling it to learn to avoid obstacles autonomously from experience. Fig. 16 depicts the simulated environment for the obstacle avoidance task, in which the robotic arm is the starting point, the blue structure is a cylindrical obstacle, and the red dot is the target. The obstacle was positioned at (−0.35, −0.45), and the target was positioned at (−0.15, −0.403, 0.22).

Fig. 17 illustrates the simulation results obtained in the obstacle avoidance task. The curves in this figure indicate that the robotic arm sometimes moved away from the target to avoid the obstacle.

Table 11 presents the MAE, RMSE, and SD of the *x*-axis, *y*-axis, *z*-axis, and angle obtained in the obstacle avoidance task by using the SAC model. The final position within the permissible error range was recorded, and the distance error from the target to this position was calculated using the distance formula (table12).

4.2. Hyperparameter optimization

This study employed a genetic algorithm (GA) to optimize the hyperparameters of the SAC model in order to improve learning efficiency in robotic arm control. Through a relatively simple search mechanism, the GA iteratively selects the optimal set of hyperparameters that minimize the distance between the robotic arm and the target point. In each generation, a population of candidate hyperparameter sets is maintained, and their fitness values are determined on the basis of the minimum distance achieved by the robotic arm after SAC training. In this study, the initial values of the hyperparameters were set randomly. The key hyperparameters used in this study are outlined as follows: learning rate, batch size, discount factor, target network update rate, and entropy coefficient. The learning rate determines the step size for parameter updates and influences convergence speed and stability. An excessive learning rate may cause training instability, whereas an insufficient learning rate can lead to slow convergence. The batch size controls the number of data samples used in each training update. Larger batch sizes result in more stable gradient updates, whereas smaller batch sizes may enhance exploration ability. The discount factor (γ) affects the long-term reward in RL and determines the model's prioritization of future rewards. A high γ value encourages the model to focus on long-term rewards, whereas a low γ value causes the model to become more sensitive to short-term feedback. The target network update rate (τ) regulates the update speed of the target network in SAC. A small τ value prevents drastic changes and improves stability, whereas a large τ value accelerates learning progress. The entropy coefficient controls policy randomness, enabling the model to explore the action space more extensively during early training. A high entropy coefficient encourages more exploration, whereas a low entropy coefficient emphasizes deterministic decision-making. During GA evolution, the top one-third of individuals with the highest fitness values are directly retained for the next generation, whereas the remaining individuals are subjected to crossover and mutation operations to generate new hyperparameter combinations. The crossover mechanism selects parameters from high-fitness individuals to create new candidates, whereas the mutation process randomly adjusts specific parameters (e.g., modifying the learning rate or entropy coefficient) to enhance search diversity. The aforementioned process is repeated until a predefined number of generations is reached, and the individual with the highest fitness is selected as the optimal hyperparameter set. In contrast to conventional hyperparameter tuning methods such as grid search or Bayesian optimization, the GA leverages an adaptive selection and evolutionary mechanism to

Table 13
Ranges and final values of the optimized hyperparameters of the SAC model.

	Range	Final value
Learning rate	1e-10–0.79	0.008
Batch size	1, 16, 128, 512	512
γ	0.50,~0.9999	0.89
τ	0.001,~0.02	0.012
Entropy Coefficient	1e-10,~1.0e0	1.16e-09

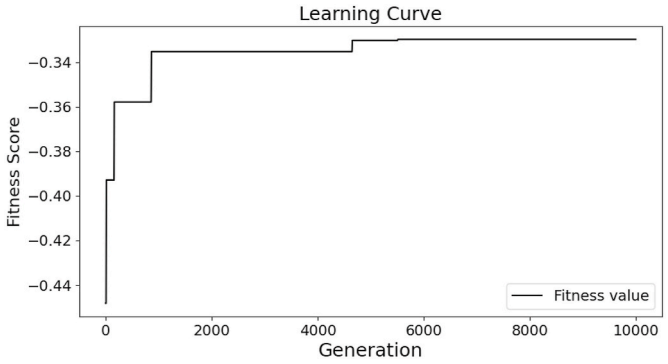


Fig. 18. Post training optimized learning curve of the GA.

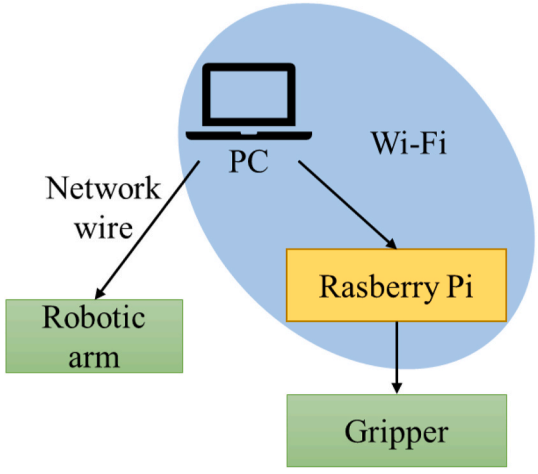


Fig. 19. Schematic of the hardware connections in the physical testing.

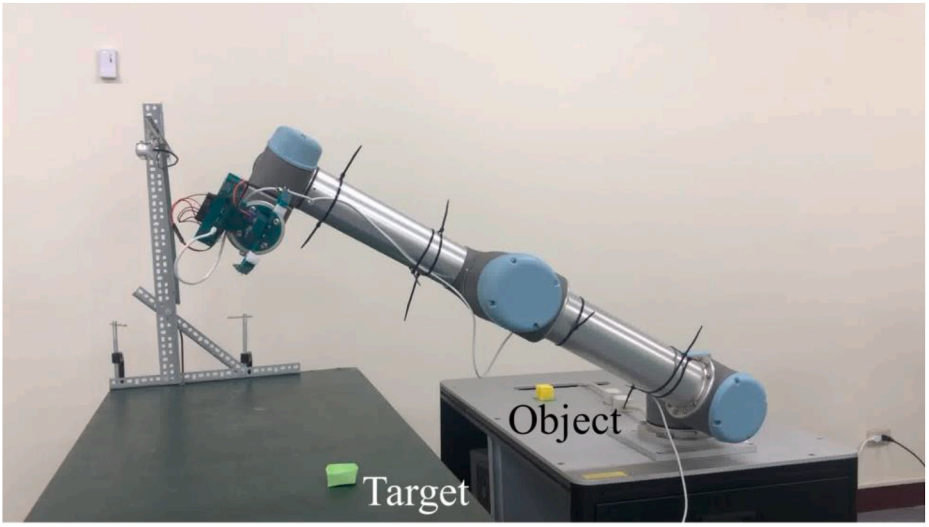


Fig. 20. Setup of the point tracking experiment.

efficiently explore high-dimensional parameter spaces, thus enabling the rapid discovery of optimal parameter combinations with limited computational resources. Furthermore, the population-based search mechanism of the GA enables it to avoid local optima, thereby ensuring a more stable training process for the SAC model in robotic arm environments.

Fig. 18 displays the post training learning curve of the GA, and Table 13 presents the ranges and final values of the optimized hyperparameters of the SAC model. During the GA-driven hyperparameter optimization process, the fitness function was defined as the negative distance to the target point, ensuring that solutions minimizing this distance were favored; thus, a higher fitness value indicated a smaller distance between the robotic arm and the target. At a high fitness value, the learning process of the SAC model stabilized, which effectively reduced the distance between the robotic arm and the target point. The final best fitness score was −0.33, and the corresponding optimal hyperparameter configuration confirmed that the GA successfully enhanced the learning efficiency and stability of the SAC model. During GA training, individuals with high fitness values were retained, whereas crossover and mutation mechanisms were applied to individuals with low fitness values to enhance search diversity and promote the exploration of the global optimal solution. The hyperparameter values presented in Table 13 indicate that the adaptive selection mechanism of the GA effectively improved the RL performance by optimizing the hyperparameters of the SAC model.

4.3. Physical testing

Physical testing was performed to validate the effectiveness of the trained SAC model in practical applications. Fig. 19 displays a schematic of the hardware connections in the physical test. The robotic arm was controlled using a computer through a network wire, and the gripper of the robot was controlled using Raspberry Pi, which was wirelessly connected to the main computer. The setup of the point tracking experiment, including the positions of the target and the object to be picked, is depicted in Fig. 20.

Fig. 21 illustrates the experimental point tracking process. During the point tracking experiment, the position and angle of the end point were determined using forward kinematics according to the motor values of each joint (Fig. 22). The graphs displayed in Fig. 22 provide insight into the process involving the robotic arm arriving at the object position, transporting the object, and then releasing the object at the target position. The high precision obtained at each point throughout this process verified the feasibility of applying the adopted model to a robotic arm in practical applications. Table14 presents the final positions of the end point of the robotic arm and the target point as well as the distance error, which was calculated from the errors along the x-axis, y-axis, and z-axis. This table indicates that the distance errors between the end point and the object picking position, midway position, and target release position (i.e., the three target points) were 0.0101, 0.0139, and 0.0215 m, respectively; the corresponding angular errors were 4.139°, 1.148°, and 3.777°, respectively.

The setup of the obstacle avoidance experiment is depicted in Fig. 23, in which the positions of the target, object to be picked, and obstacle are marked. Fig. 24 displays images of the obstacle avoidance experiment from different angles to offer a clear view of the process.

During the obstacle avoidance experiment, positions and angles were determined using forward kinematics according to the motor values of each joint (Fig. 25). The results obtained in this experiment matched the corresponding simulation results.

Table 15 presents the final positions of the end point and target point in the obstacle avoidance experiment; the distance error and angular error were 0.0185 m and 7.772°, respectively.

5. Conclusion

In this study, RL models based on the SAC, A2C, PPO, and TRPO algorithms were constructed to train a robotic arm and perform point tracking. Environmental observations, robot actions, and a reward function were designed, following which a robotic arm simulation environment was established. The aforementioned four models were used to perform a point tracking task in this environment. Among these models, the SAC model exhibited the best performance in the aforementioned task and was therefore used to perform an obstacle avoidance task. Several parameters related to the simulation environment, such as the environmental observations and reward function, were redefined to facilitate the learning of obstacle avoidance. The design of the reward function had a considerable effect on training outcomes.

Because of limitations associated with the hardware architecture of robotic arms, the singularity problems encountered in forward and

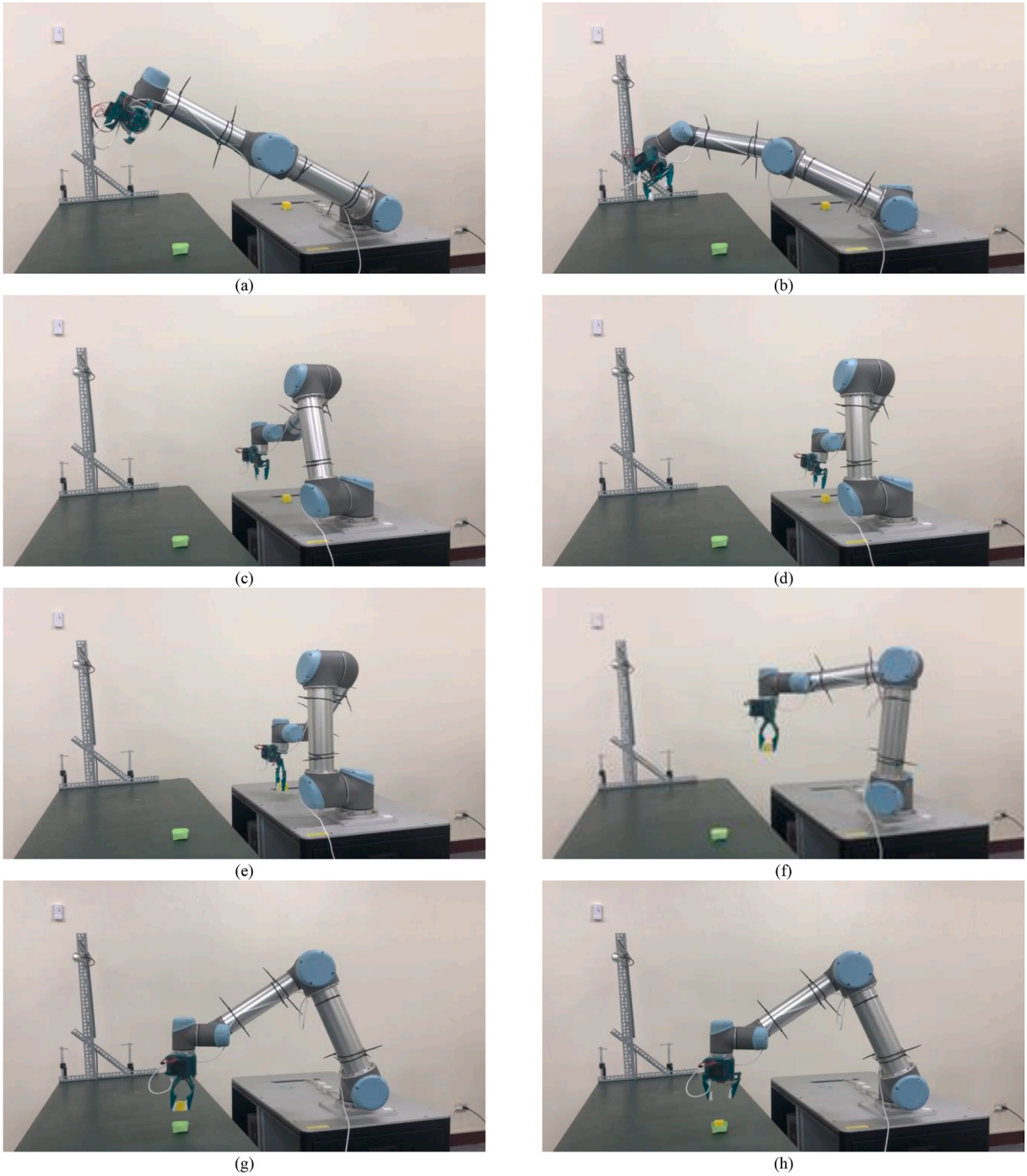


Fig. 21. Illustration of the point tracking experiment: (a) initialization of the environment and robotic arm, (b)–(c) movement of the robotic arm into the gripping position, (d) arrival of the gripper at the position of the object to be picked, (e) initiation of the picking action by the gripper, (f)–(g) movement of the object to the next position by the arm, and (h) arrival of the arm at the target position in the required direction as well as the release of the object.

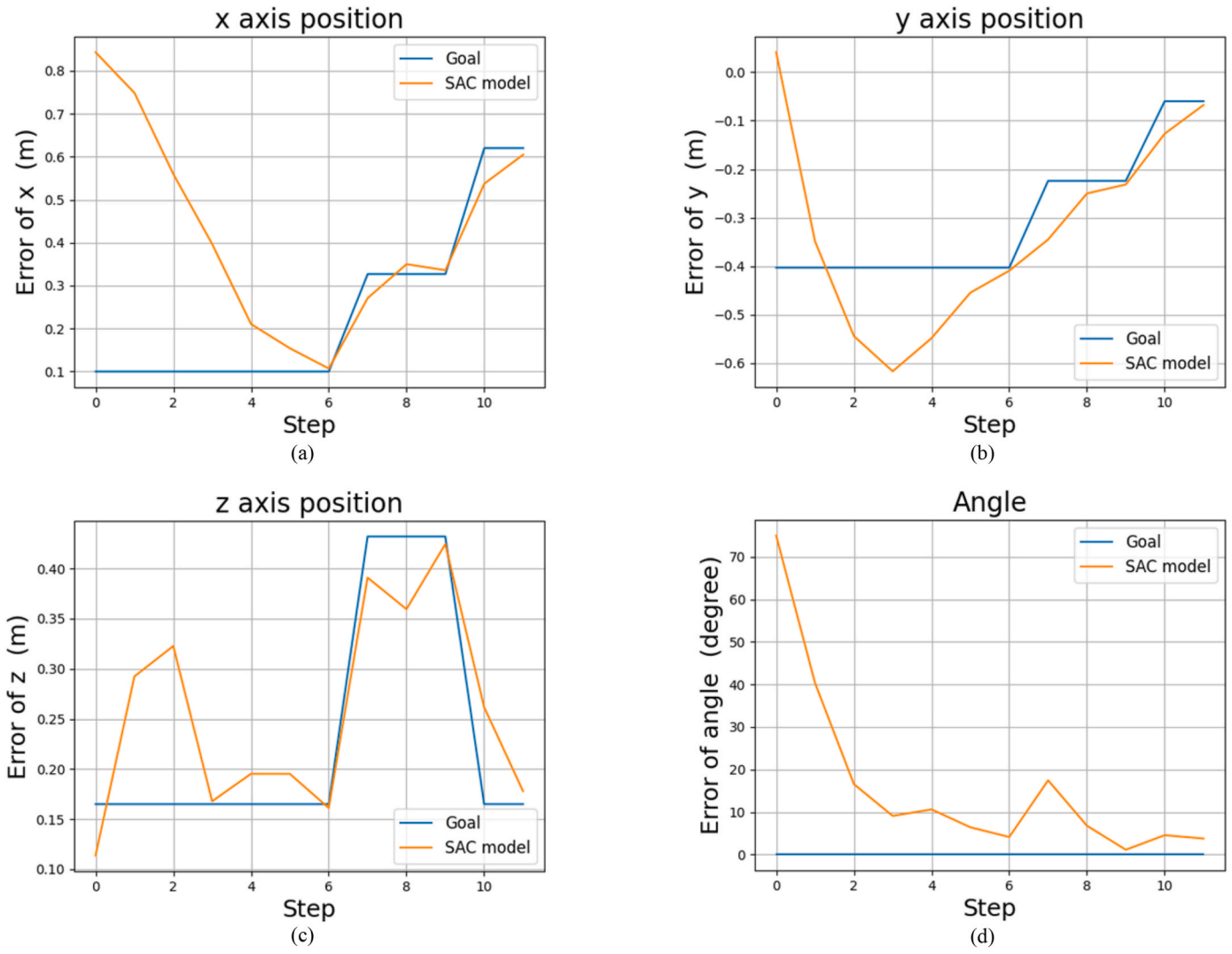


Fig. 22. Errors in the point tracking experiment: (a) *x*-axis, (b) *y*-axis, (c) *z*-axis, and (d) angular errors.

inverse kinematic control cannot be effectively solved using existing methods or algorithms. However, these problems can be overcome using the proposed robotic arm control model because this model searches for the closest position to the target point. Although the addition of constraints may result in the true optimal solution being excluded from the action space, in practical robotic arm applications, safely reaching the target within the reachable range is more important than mathematical optimality. Therefore, this study prioritized the safest solution within the reachable range as the optimal solution. Of the four adopted models, the SAC model exhibited the smallest MAE, RMSE, and SD values for its angular and coordinate errors. The SD values of the SAC model were equal to 0, which indicated the high stability and performance of this model. This model also exhibited suitable results in the simulated object avoidance task. Finally, physical testing verified the applicability of the SAC model in robotic arms for point tracking and obstacle avoidance in practical settings. The proposed model is an alternative approach to the conventional forward and inverse kinematic control of robotic arms. This model eliminates the need for complex calculations and offers higher agility in robotic arm control than do existing control models. The model can enable factories to implement intelligent operations, facilitate the adaptation of robotic arms to dynamic application environments, and reduce technical barriers for robotic arm users, thus making robotic arms more widely deployable across various fields.

Although the proposed model addresses the singularity problems encountered in kinematic control, its training process might require considerable time in practical scenarios because of its use of an RL model. Theoretically, RL models may identify suboptimal paths on account of being trapped in local optima or conducting insufficient exploration. Future research can focus on optimizing RL models to reduce their training time, thus enabling these models to adapt to dynamic environments rapidly in industrial applications. Moreover, efforts can be aimed at generating the most efficient paths, thereby reducing energy consumption.

Table 14
Final position of the end point relative to the target point during the point tracking experiment (m).

	x-axis	*y*-axis	*z*-axis	Distance error
Goal 1	0.1	−0.403	0.165	0.0101
End-Effector 1	0.1070	−0.4092	0.1612	
Goal 2	0.327	−0.224	0.432	0.0139
End-Effector 2	0.3358	−0.2316	0.4245	
Goal 3	0.62	−0.06	0.165	0.0215
End-Effector 3	0.6048	−0.0678	0.1780	

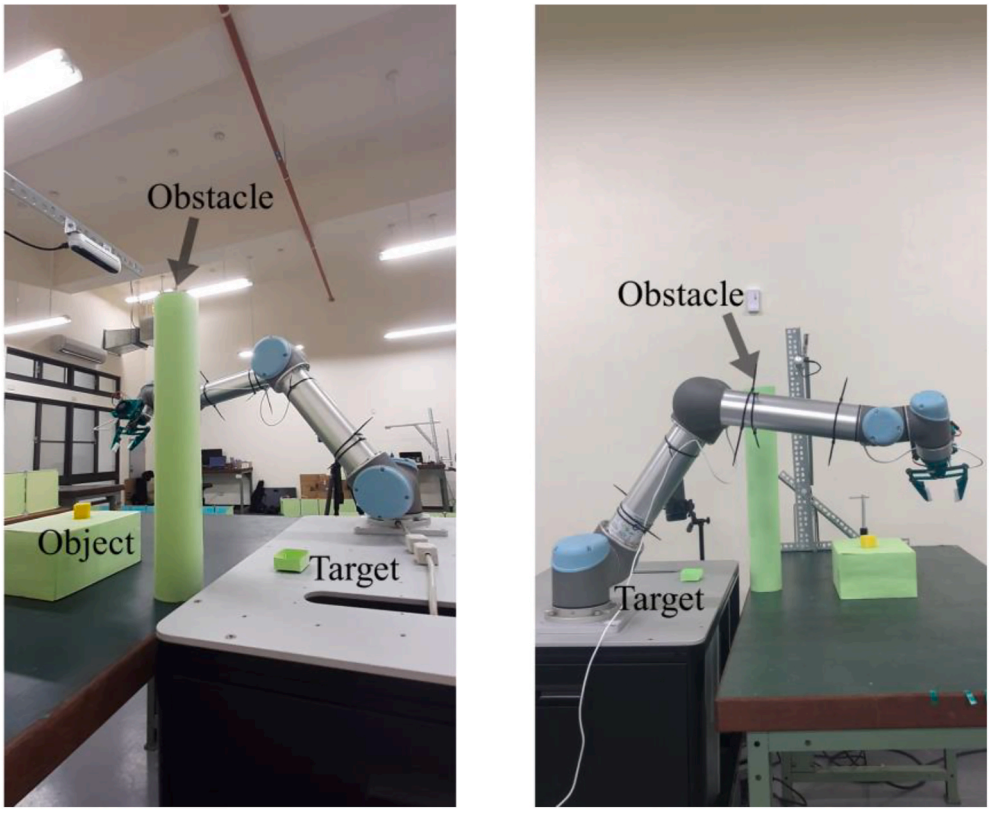


Fig. 23. Setup of the obstacle avoidance experiment.

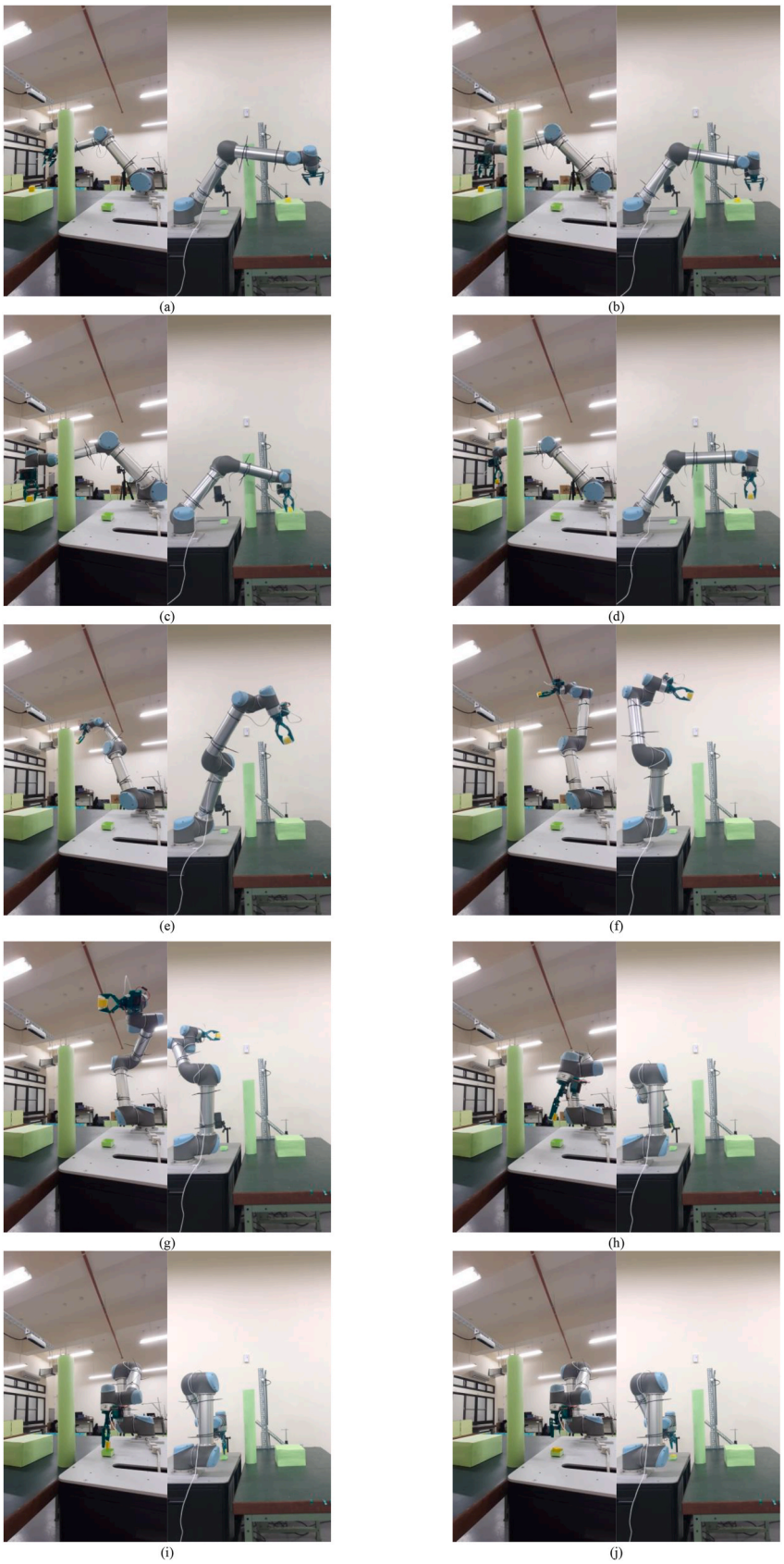


Fig. 24. Illustration of the obstacle avoidance experiment: (a) and (b) initialization of the environment and robotic arm, (c) initiation of the picking action by the gripper, (d)–(h) transportation of the picked object to the next position along the shortest path while avoiding the obstacle, (i) arrival at the target position and the direction of the object, and (j) release of the picked object.

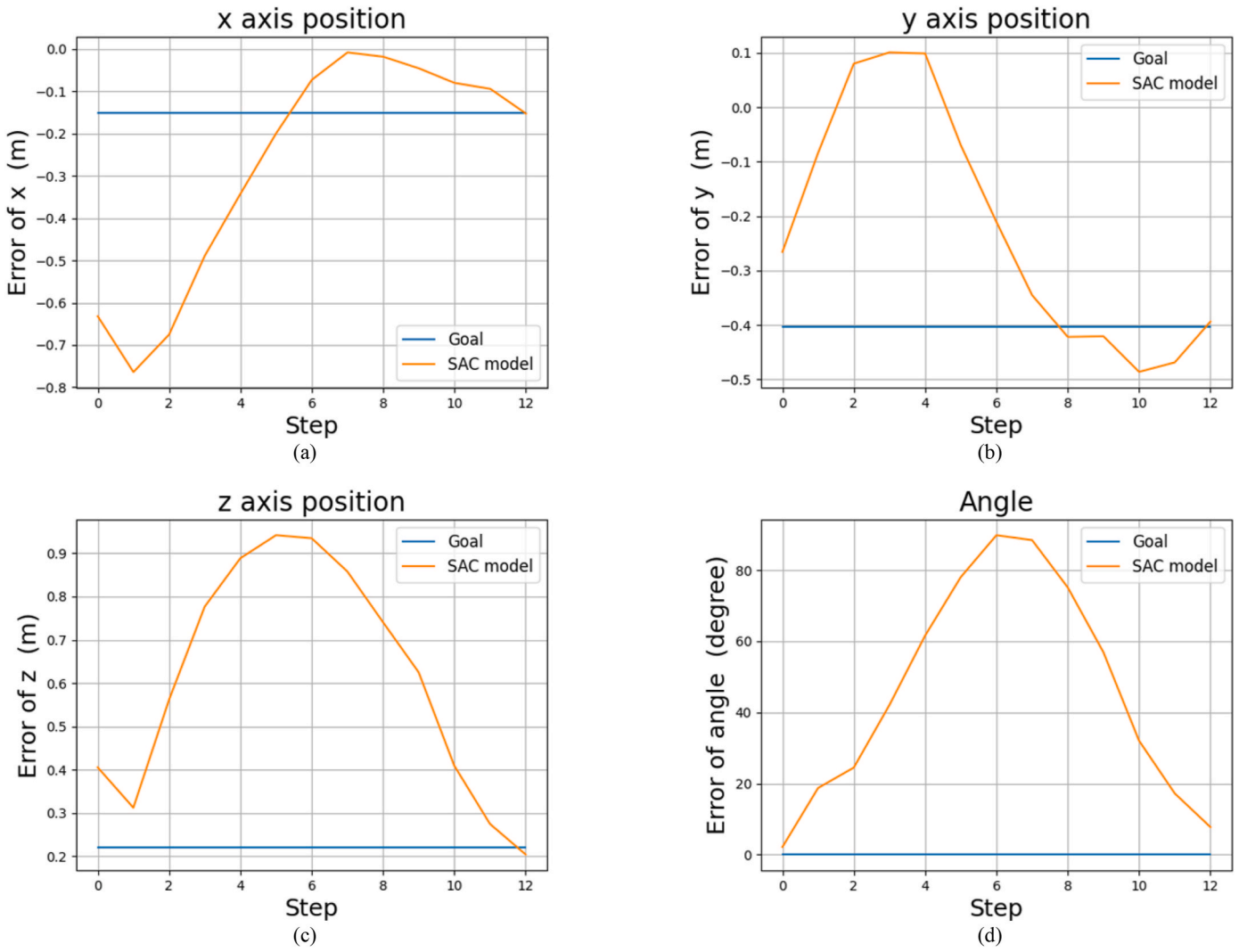


Fig. 25. Errors in the obstacle avoidance experiment: (a) *x*-axis, (b) *y*-axis, (c) *z*-axis, and (d) angular errors.

Table 15
Final position of the end point relative to the target point during the obstacle avoidance experiment (m).

	x-axis	*y*-axis	*z*-axis	Distance error
Goal	−0.15	−0.403	0.22	0.0185
End Effector	−0.1525	−0.3940	0.2041	

CRediT authorship contribution statement

Ping-Huan Kuo: Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization. **Chen-Ting Huang:** Writing – review & editing, Writing – original draft, Software, Methodology. **Chen-Wen Chang:** Writing – review & editing, Writing – original draft, Software. **Po-Hsun Feng:** Software, Writing – original draft, Writing – review & editing. **Yu-Sian Lin:** Software, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Science and Technology Council, Taiwan, under Grants NSTC 111-2218-E-194-007, NSTC 112-2218-E-194-006, MOST 111-2823-8-194-002, MOST 111-2221-E-194-052, NSTC 112-2221-E-194-032, and NSTC 113-2221-E-194-031. This work was financially partially supported by the Advanced Institute of Manufacturing with High-tech Innovations (AIM-HI) from The Featured Areas Research Center Program within the framework of the Higher Education Sprout Project by the Ministry of Education (MOE) in Taiwan.

Data availability

No data was used for the research described in the article.

References

Adibi, M., van der Woude, J., 2022. Secondary frequency control of microgrids: an online reinforcement learning approach. IEEE Trans. Automat. Control 67, 4824–4831. https://doi.org/10.1109/TAC.2022.3162550.

Araújo, J.P., Figueiredo, M.A.T., Ayala Botto, M., 2022. Control with adaptive Q-learning: a comparison for two classical control problems. Eng. Appl. Artif. Intell. 112, 104797. https://doi.org/10.1016/j.engappai.2022.104797.

Bing, Z., Brucker, M., Morin, F.O., Li, R., Su, X., Huang, K., Knoll, A., 2022. Complex robotic manipulation via graph-based hindsight goal generation. IEEE Transact. Neural Networks Learn. Syst. 33, 7863–7876. https://doi.org/10.1109/TNNLS.2021.3088947.

Boudlal, A., Khafaji, A., Elabbadi, J., 2024. Entropy adjustment by interpolation for exploration in proximal policy optimization (PPO). Eng. Appl. Artif. Intell. 133, 108401. https://doi.org/10.1016/j.engappai.2024.108401.

Du, Z., Xie, X., Qu, Z., Hu, Y., Stojanovic, V., 2024. Dynamic event-triggered consensus control for interval type-2 fuzzy multi-agent systems. IEEE Transactions on Circuits and Systems I: Regular Papers 71, 3857–3866. https://doi.org/10.1109/TCSI.2024.3371492.

el Zaatari, S., Wang, Y., Li, W., 2021. Reinforcement learning based optimization for cobot's path generation in collaborative tasks. In: 2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design (CSCWD). IEEE, pp. 975–980. https://doi.org/10.1109/CSCWD49262.2021.9437874.

Gafni, T., Cohen, K., 2021. Learning in restless multiarmed bandits via adaptive arm sequencing rules. IEEE Trans. Automat. Control 66, 5029–5036. https://doi.org/10.1109/TAC.2020.3043431.

Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018a. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., Levine, S., 2018b. Soft Actor-Critic Algorithms and Applications.

Hu, Z., Zheng, Y., Pan, J., 2023. Grasping living objects with adversarial behaviors using inverse reinforcement learning. Robot. IEEE Trans. 39, 1151–1163. https://doi.org/10.1109/TRO.2022.3226108.

Jeong, I.B., Ko, W.R., Park, G.M., Kim, D.H., Yoo, Y.H., Kim, J.H., 2017. Task intelligence of robots: neural model-based mechanism of thought and online motion planning. IEEE Trans Emerg Top Comput Intell 1, 41–50. https://doi.org/10.1109/TETCI.2016.2645720.

Joshua Achiam, 2020. Spinning up documentation-trust region policy optimization [WWW Document]. OpenAI. URL. https://spinningup.openai.com/en/latest/algorithms/trpo.html#saved-model-contents. (Accessed 7 January 2023).

Konda, V., Tsitsiklis, J., 1999. Actor-critic algorithms. In: Solla, S., Leen, T., Müller, K. (Eds.), Advances in Neural Information Processing Systems. MIT Press.

Koprinkova-Hristova, P., 2010. Backpropagation through time training of a neuro-fuzzy controller. Int. J. Neural Syst. 20, 421–428. https://doi.org/10.1142/S0129065710002504.

Lan, X., Hu, Y., Xie, Y., Meng, X., Zhang, Y., Pan, Q., Ding, Y., 2023. Innovation design oriented functional knowledge integration framework based on reinforcement learning. Adv. Eng. Inform. 58, 102122. https://doi.org/10.1016/j.aei.2023.102122.

Li, H., Zhang, Y., Wen, H., Zhu, Y., Zhao, D., 2024. Stabilizing diffusion model for robotic control with dynamic programming and transition feasibility. IEEE Transactions on Artificial Intelligence 5, 4585–4594. https://doi.org/10.1109/TAI.2024.3387401.

Luipers, D., Kaulen, N., Chojnowski, O., Schneider, S., Richert, A., Jeschke, S., 2022. Robot control using model-based reinforcement learning with inverse kinematics. In: 2022 IEEE International Conference on Development and Learning (ICDL). IEEE, pp. 244–249. https://doi.org/10.1109/ICDL53763.2022.9962215.

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous Methods for Deep Reinforcement Learning.

Mohamadi, N., Niaki, S.T.A., Taher, M., Shavandi, A., 2024. An application of deep reinforcement learning and vendor-managed inventory in perishable supply chain management. Eng. Appl. Artif. Intell. 127, 107403. https://doi.org/10.1016/j.engappai.2023.107403.

Pasandi, V., Sadeghian, H., Keshmiri, M., Pucci, D., 2022. An integrated programmable CPG with bounded output. IEEE Trans. Automat. Control 67, 4658–4673. https://doi.org/10.1109/TAC.2022.3166715.

Popova, S., Koprinkova, P., Patarinska, T., 2003. Neural network based biomass and growth rate estimation aimed to control of a chemostat microbial cultivation. Appl. Artif. Intell. 17, 345–360. https://doi.org/10.1080/713827143.

Precup, R.-E., Roman, R.-C., Safaei, A., 2021. Data-Driven Model-free Controllers. CRC Press, Boca Raton. https://doi.org/10.1201/9781003143444.

Roman, R.-C., Precup, R.-E., Hedrea, E.-L., Preitl, S., Zamfirache, I.A., Bojan-Dragos, C.-A., Petriu, E.M., 2022. Iterative feedback tuning algorithm for tower crane systems. Procedia Comput. Sci. 199, 157–165. https://doi.org/10.1016/j.procs.2022.01.020.

Saab, S.S., Ghanem, P., 2018. A multivariable stochastic tracking controller for robot manipulators without joint velocities. IEEE Trans. Automat. Control 63, 2481–2495. https://doi.org/10.1109/TAC.2017.2771154.

Schulman, J., Levine, S., Moritz, P., Jordan, M.I., Abbeel, P., 2015. Trust Region Policy Optimization.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal Policy Optimization Algorithms.

Shakouri, A., Assadian, N., 2022. Prescribed-time control for perturbed euler-Lagrange systems with obstacle avoidance. IEEE Trans. Automat. Control 67, 3754–3761. https://doi.org/10.1109/TAC.2021.3106882.

Sun, F., Chen, Y., Wu, Y., Li, L., Ren, X., 2022. Motion planning and cooperative manipulation for mobile robots with dual arms. IEEE Trans Emerg Top Comput Intell 6, 1345–1356. https://doi.org/10.1109/TETCI.2022.3146387.

Tang, F., Feng, Z., Li, Y., Yang, C., Sun, B., 2023. A constrained multi-objective deep reinforcement learning approach for temperature field optimization of zinc oxide rotary volatile kiln. Adv. Eng. Inform. 58, 102197. https://doi.org/10.1016/j.aei.2023.102197.

Tao, H., Zheng, J., Wei, J., Paszke, W., Rogers, E., Stojanovic, V., 2023. Repetitive process based indirect-type iterative learning control for batch processes with model uncertainty and input delay. J. Process Control 132, 103112. https://doi.org/10.1016/J.JPROCONT.2023.103112.

Tao, Hongfeng, Zheng, Yuechang, Wang, Yue, Qiu, Jier, Stojanovic, Vladimir, 2024. Enhanced Feature Extraction YOLO Industrial Small Object Detection Algorithm Based on Receptive-Field Attention and Multi-Scale Features, vol. 35, 105023. https://doi.org/10.1088/1361-6501/ad633d.

Wang, J.X., Kurth-Nelson, Z., Kumaran, D., Tirumala, D., Soyer, H., Leibo, J.Z., Hassabis, D., Botvinick, M., 2018. Prefrontal cortex as a meta-reinforcement learning system. Nat. Neurosci. 21, 860–868. https://doi.org/10.1038/s41593-018-0147-8.

Wang, C., Cui, X., Zhao, S., Zhou, X., Song, Y., Wang, Y., Guo, K., 2024. Enhancing vehicle ride comfort through deep reinforcement learning with expert-guided soft-hard constraints and system characteristic considerations. Adv. Eng. Inform. 59, 102328. https://doi.org/10.1016/J.AEI.2023.102328.

Wen, J., Wang, F., 2024. Stable levitation of single-point levitation systems for maglev trains by improved cascade control. Rom. J. Inf. Sci. Technol. 27, 348–361. https://doi.org/10.59277/ROMJIST.2024.3-4.08.

Wigren, T., Lau, K., Delgado, R.A., Middleton, R.H., 2020. Globally stable delay alignment for feedback control over wireless multipoint connections. IEEE Trans Control Netw Syst 7, 1633–1642. https://doi.org/10.1109/TCNS.2020.2989786.

Yang, Y., Ni, Z., Gao, M., Zhang, J., Tao, D., 2022. Collaborative pushing and grasping of tightly stacked objects via deep reinforcement learning. IEEE/CAA Journal of Automatica Sinica 9, 135–145. https://doi.org/10.1109/JAS.2021.1004255.

Zamfirache, I.A., Precup, R.-E., Petriu, E.M., 2024. Adaptive reinforcement learning-based control using proximal policy optimization and slime mould algorithm with experimental tower crane system validation. Appl. Soft Comput. 160, 111687. https://doi.org/10.1016/j.asoc.2024.111687.

Zamfirache, I.A., Precup, R.-E., Petriu, E.M., 2025. Safe reinforcement learning-based control using deep deterministic policy gradient algorithm and slime mould algorithm with experimental tower crane system validation. Inf. Sci. 692, 121640. https://doi.org/10.1016/j.ins.2024.121640.

Zhang, M., Tian, G., Cui, Y., Zhang, Y., Xia, Z., 2023. Hierarchical semantic knowledge-based object search method for household robots. IEEE Trans Emerg Top Comput Intell. https://doi.org/10.1109/TETCI.2023.3297838.